

SIDDHARTH INSTITUTE OF ENGINEERING & TECHNOLOGY

(AUTONOMOUS)



(Approved by AICTE, New Delhi & Affiliated to JNTUA, Ananthapuramu)
(Accredited by NBA for Civil, EEE, Mech., ECE & CSE)
(Accredited by NAAC with 'A+' Grade)

QUESTION BANK (DESCRIPTIVE)

Subject with Code: Digital Logic and Computer Organization (23CS0506)

Course & Branch: B.Tech – CSM, CAD, CAI

Year & Semester: II - B. Tech. & II-Semester

Regulation: R23

UNIT-I PART-

A (2 MARKS)

1(a) What is binary number system?

Binary Number System: The binary number system is a way of representing numbers using only two digits: 0 and 1. It's also called the base-2 number system.

Unlike the decimal system (base-10), which uses digits 0 through 9, binary only uses two symbols. Each digit in a binary number is called a bit (short for binary digit).

Binary is the fundamental language of computers because digital electronics use two states: on (1) And off (0)

Example: Convert 4 in binary. Solution:

4 in binary is $(100)_2$.

(b) What is an Excess-3 code?

Excess-3 Code: The excess-3 code (or XS3) is a non-weighted code used to express code used to express decimal numbers. It is a self-complementary binary coded decimal (BCD) code and numerical system which has biased representation. It is particularly significant for arithmetic operations as it overcomes shortcoming encountered while using 8421 BCD code to add two decimal digits whose sum exceeds 9. Excess-3 arithmetic uses different algorithm than normal non-biased BCD or binary positional number system.

(C) What are the basic properties of Boolean algebra?

The three basic properties of Boolean algebra are the commutative, associative, and distributive properties. The commutative property states that the order of the variable does not matter in OR and AND operations. The associative property states that parentheses can be moved when two OR or two AND operations are next to each other. The distributive property describes what happens when an AND is distributed over an OR operation and what happens when an OR is distributed over an AND operation.



Edit with WPS Office

(D) List out the names of basic logical operators.

There are three logical operators: AND, OR and NOT.

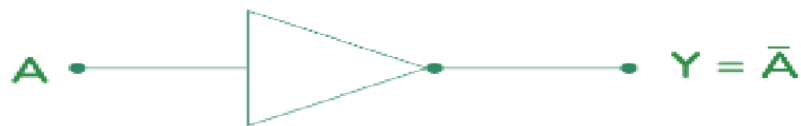
2-Input OR Gate



Truth Table

Input A	Input B	Output
0	0	0
0	1	1
1	0	1
1	1	1

NOT Gate



Truth Table

A (Input)	$Y = \bar{A}$ (Output)
0	1
1	0

NOT Gate With Truth Table

2- Input AND Gate



Truth Table

A (Input 1)	B (Input 2)	$X = (A.B)$
0	0	0
0	1	0
1	0	0
1	1	1

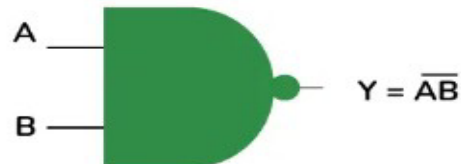


(E) List the names of universal gates with symbols

NAND and NOR gates are the universal gates.

A NAND gate is a combination of an AND gate followed by a NOT gate. It outputs a 0 only when all its inputs are 1; otherwise, it outputs 1.

Symbol



Truth Table

Input A	Input B	$Y = \overline{AB}$
0	0	1
0	1	1
1	0	1
1	1	0

NOR Gate

A NOR gate is a combination of an OR gate followed by a NOT gate. It outputs a 1 only when all its inputs are 0; otherwise, it outputs 0.

Symbol



Truth Table

Input A	Input B	$0 = (A + B)'$
0	0	1
0	1	0
1	0	0
1	1	0



2.(a) Differentiate between floating point representation and fixed-point representation.

Floating Point Representation	Fixed Point Representation
1. A Formula tic representation of real numbers as an approximation so as to support a tradeoff between range and precision.	1. A representation of real data type for a number that has a fixed number of digits after the radix part.
2. Used to represent a wide range of values	2. Used to represent a limited range of values
3. Lower Performance	3. Higher Performance
4. More flexible	4. Less flexible
5. Over all the code is clearer and easy to read	5. This results in more lines of code which are consequently more difficult to read.
6. Numerical stability without double precision values is hard to ensure	6. Numerical stability can be determined clearly.

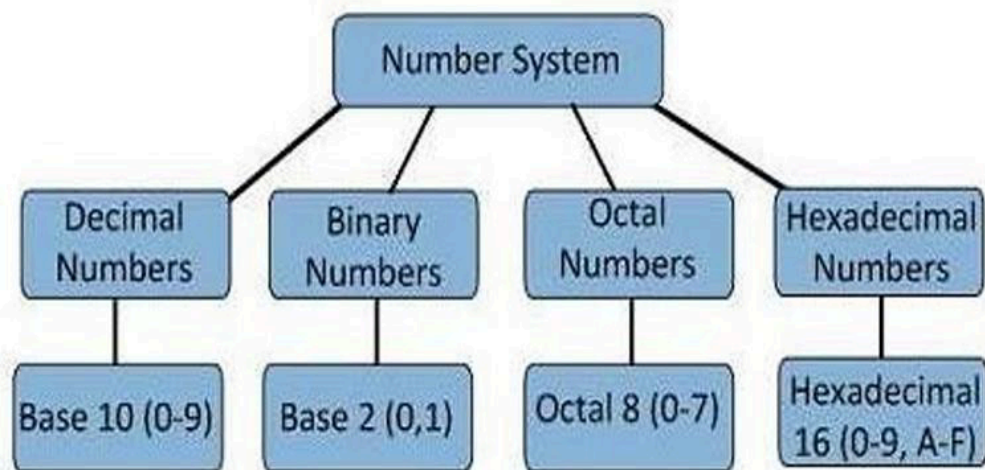
(b) What is number system? Explain the different types of number systems

Definition: In digital electronics, the number system is used for representing the information. The number system has different bases and the most common of them are the decimal, binary, octal, and hexadecimal.

The base or radix of the number system is the total number of the digit used in the number system. Ex: Suppose if the number system representing the digit from 0 – 9 then the base of the system is the 10.

Types of Number Systems:

- Decimal Number System
- Binary Number System
- Octal Number System
- Hexadecimal Number System



1. **Decimal Number Systems:** The number system is having digit 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 is known as a decimal number system because total ten digits are involved. The base of the decimal



number system is 10.

2. Binary Number Systems: The modern computers do not process decimal number; they work with another number system known as a binary number system which uses only two digits 0 and 1. The base of binary number system is 2 because it has only two digit 0 and 1.

3. Octal Number system: The Octal system is base 8. Therefore, it requires 8 digits. The base of octal number system is 8 because it has only two digit 0 to 7.

Octal Digit Value	Binary Equivalent
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111

4. Hexadecimal Number system: The hexadecimal system is base 16. Therefore, it requires 16 different symbols. The values 0 through 9 are used, along with the letters A through F, which represent the decimal values 10 through 15.

binary	Hexadecimal
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	8
1001	9
1010	A
1011	B
1100	C
1101	D
1110	E
1111	F



3.(a) Convert the $(555)_{10}$ into binary, octal and Hexadecimal number systems.

1. $(555)_{10} \rightarrow ()_2$

2		555	
2		277	- 1
2		138	- 1
2		69	- 0
2		34	- 1
2		17	- 0
2		8	- 1
2		4	- 0
2		2	- 0
2		1	- 0

$(1000101011)_2$

Decimal to octal :-

$(555)_{10} \rightarrow ()_8$

8		555	
8		69	- 3
8		8	- 5
8		1	- 0

$(555)_{10} \rightarrow (1053)_8$

Decimal to Hexa decimal :-

$(555)_{10} \rightarrow ()_{16}$

16		555	
16		34	- 11
16		2	- 2

$(555)_{10} \rightarrow (22B)_{16}$



(b) Convert the following into binary to decimal i) $(1101.1)_2$ ii) $(1100.001)_2$

1. $(1101.1)_2$

$$\begin{array}{ccccccc} 1 & 1 & 0 & 1 & . & 1 & \\ 2^3 & 2^2 & 2^1 & 2^0 & & 2^{-1} & \end{array}$$

$$1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1}$$

$$8 + 4 + 0 + 0 + 0.5$$

$$(13.5)_{10}$$

2. $(1100.001)_2$

$$\begin{array}{ccccccc} 1 & 1 & 0 & 0 & . & 0 & 0 & 1 \\ 2^3 & 2^2 & 2^1 & 2^0 & & 2^{-1} & 2^{-2} & 2^{-3} \end{array}$$

$$1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 0 \times 2^0 + 0 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3}$$

$$8 + 4 + 0 + 0 + 0 + 0 + 0 + 0.125$$

$$12 + 0.125$$

$$(12.125)_{10}$$

4.(a) Convert the following into Decimal into Hexa decimal

i) $(5386.34)_{10}$ ii) $(214.35)_{10}$

i) $(5386.34)_{10}$

$$\begin{array}{r} 16 \overline{) 5386} \\ 16 \overline{) 336} - 10 \\ 16 \overline{) 21} - 0 \\ 16 \overline{) 1} - 5 \end{array}$$

$$(150A)_{16} = 5386$$

fractional part $(0.34)_{10}$

$$0.34 \times 16 = 5.44 = 5$$

$$0.44 \times 16 = 7.04 = 7$$

$$0.04 \times 16 = 0.64 = 0$$

$$(150A.570)_{16} = (5386.34)_{10}$$

ii) $(214.35)_{10}$

integer part $(214)_{10}$

$$\begin{array}{r} 16 \overline{) 214} \\ 16 \overline{) 13} - 6 \end{array}$$

$$(D6)_{16} = (214)_{10}$$

fractional part $(0.35)_{10}$

$$0.35 \times 16 = 5.6 = 5$$

$$0.6 \times 16 = 9.6 = 9$$

$$0.6 \times 16 = 9.6 = 9$$

$$(214.35)_{10} = (D6.599)_{16}$$



(b) Represent signed numbers from +7 to -8 using different ways of representation.

Decimal	Signed Magnitude	1's Complement	2's Complement
+7	0111	0111	0111
+6	0110	0110	0110
+5	0101	0101	0101
+4	0100	0100	0100
+3	0011	0011	0011
+2	0010	0010	0010
+1	0001	0001	0001
0	0000	0000	0000
-0	1000	1111	(not distinct)
-1	1001	1110	1111
-2	1010	1101	1110
-3	1011	1100	1101
-4	1100	1011	1100
-5	1101	1010	1011
-6	1110	1001	1010
-7	1111	1000	1001
-8	(not representable)	(not representable)	1000

5. Explain the Binary codes with examples.

Binary codes: The digital data is represented, stored and transmitted as groups of binary digits. The group of bits, also known as binary code, represent both numbers and letters of the alphabets as well as many special characters and control functions. They are classified as numeric or alphanumeric. Numeric codes are used to represent numbers. Alphanumeric codes are used to represent characters: Alphabetic letters and Numerals.

Classification of Binary Codes:

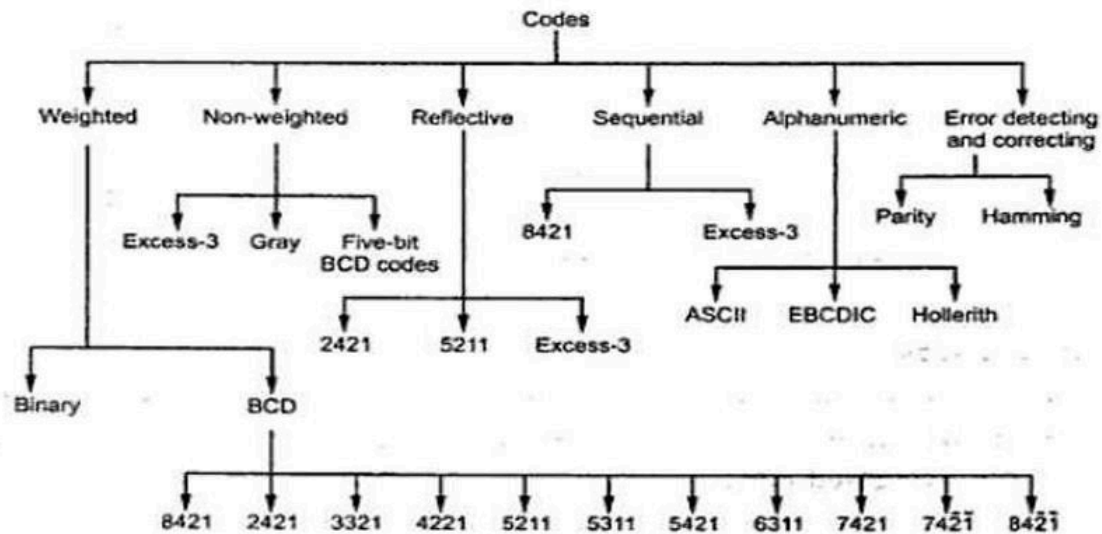
The different binary codes can be classified as

- Weighted Codes



Edit with WPS Office

- Non-Weighted Codes
- Reflective Codes
- Sequential Codes
- Alphanumeric Codes
- Error Detecting and Correcting Codes



Binary codes block diagram

Weighted Codes: In weighted codes, each digit position of the number represents a specific weight. For example, in decimal code, if number is 567 then weight of 5 is 100, weight of 6 is 10 and weight of 7 is 1. In weighted binary codes each digit has a weight 8421. The codes 8421, 2421 and 5211 are all weighted codes.

Non-weighted Codes: Non-weighted codes are not assigned with any weight to each digit position, i.e. each digit position with in number is not assigned fixed value. Excess-3 and gray codes are the non-weighted codes.

Reflective codes: A code is reflective when the code is self-complementing. In other words, when the code for 9 is the complement the code for 0, 8 for 1, 7 for 2, 6 for 3 and 5 for 4. 2421BCD, 5421BCD and Excess-3 code are reflective codes.

Sequential codes: In sequential codes, each succeeding 'code is one binary number greater than its preceding code. The 8421 and excess-3 are sequential, whereas the 2421 and 5211 codes are not.

Alphanumeric codes: The codes which consists of both numbers and alphabetic characters are called alphanumeric codes. The most commonly used alphanumeric codes are: American Standard Code for Information Interchange (ASCII), Extended Binary Coded Decimal Interchange Code (EBCDIC) and Hollerith code.

Error defecting and correcting codes: Codes which allow only error detection are called error



detecting codes and Codes which allow error detection and correction are called error detecting and correcting codes. Hamming code is the mostly commonly used error detecting and correcting code.

6.(a) List the Basic Logic functions.

A logic gate is a device that acts as a building block for digital circuits. They perform basic logical functions that are fundamental to digital circuits. Most electronic devices we use today will have some form of logic gates in them. For example, logic gates can be used in digital electronics such as smartphones and tablets or in memory devices.

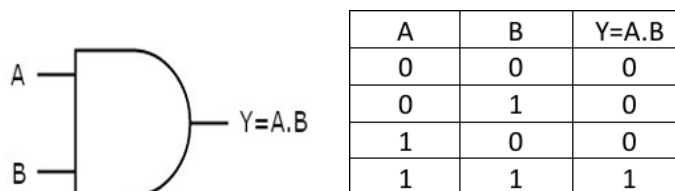
Basic logic gates

There are seven basic logic gates: AND, OR, XOR, NOT, NAND, NOR and XNOR.

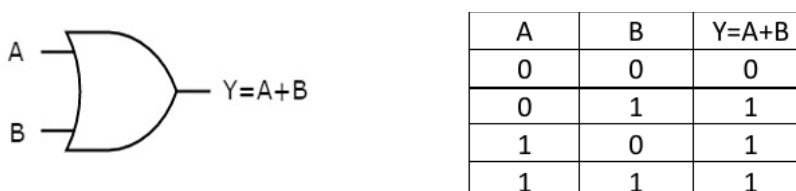
(b) Explain about Basic Logic gates with symbols and truth table.

A basic logic gate is an elementary building block that takes one or more inputs and produces a single output based on a specific truth table. There are only three fundamental logic gates - AND, OR and NOT - that can be combined in different configurations to construct more complex digital circuits and systems.

AND Gates: An AND gate is a digital circuit that has two or more inputs and produces an output, which is the logical AND of all those inputs. It is optional to represent the Logical AND with the symbol '·'.



OR gate: An OR gate is a digital circuit that has two or more inputs and produces an output, which is the logical OR of all those inputs. This logical OR is represented with the symbol '+'. .



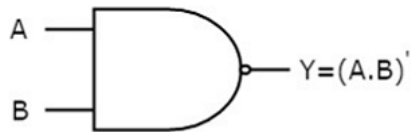
NOT gate: A NOT gate is a digital circuit that has single input and single output. The output of NOT gate is the logical inversion of input. Hence, the NOT gate is also called as inverter.



7.(a) Explain about Universal Logic gates with symbols and truth table.

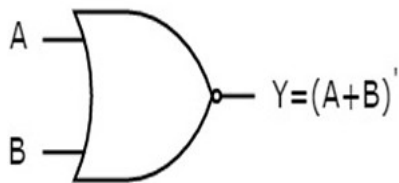
NAND and NOR gates are the universal gates.

NAND gate: NAND gate is a digital circuit that has two or more inputs and produces an output, which is the inversion of logical AND of all those inputs.



A	B	$Y = (A.B)'$
0	0	1
0	1	1
1	0	1
1	1	0

NOR gate: NOR gate is a digital circuit that has two or more inputs and produces an output, which is the inversion of logical OR of all those inputs.



A	B	$Y = (A+B)'$
0	0	1
0	1	0
1	0	0
1	1	0

(b) Prove the following identities:

$$A'B'C' + A'BC' + AB'C' + ABC' = C'AB$$

$$+ ABC + A'B + AB'C = B + AC$$

$$(i) A'B'C' + A'BC' + AB'C' + ABC' = C'$$

$$A'B'C' + A'BC' + AB'C' + ABC'$$

$$\rightarrow A'C'(B' + B) + AC'(B' + B)$$

$$\therefore B' + B = 1 \quad [\text{Apply complement formula}]$$

$$\rightarrow A'C'(1) + AC'(1)$$

$$\rightarrow A'C' + AC'$$

$$\rightarrow C'(A' + A)$$

$$\therefore A' + A = 1 \quad [\text{Apply complement formula}]$$

$$\rightarrow C'$$

We proved that :-

$$\therefore A'B'C' + A'BC' + AB'C' + ABC' = C'$$



$$(ii) \quad AB + ABC + A'B + AB'C = B + AC$$

$$\begin{aligned} \text{Sol:} \quad & AB + ABC + A'B + AB'C \\ & \quad \quad \quad \uparrow \quad \quad \quad \uparrow \\ \Rightarrow & B(A + A') + ABC + AB'C \end{aligned}$$

$$\therefore A + A' = 1$$

$$\Rightarrow B(1) + AC(B + B')$$

$$\rightarrow B + AC$$

$$\therefore B + B' = 1$$

$\therefore$ we proved that

$$AB + ABC + A'B + AB'C = B + AC$$

8.(a) Simplify the following Boolean Expressions:

$A'C' + ABC + AC' + AB$ to three literals.

$A'B(D' + C'D) + B(A + A'CD)$ to one literal.

$$\text{Sol:} \quad A'C' + ABC + AC' + AB$$

$$\Rightarrow C'(A' + A) + ABC + AB$$

$$\Rightarrow A' + A = 1$$

$$\Rightarrow C' + AB(1 + C)$$

$$\Rightarrow 1 + C = 1$$

$$\Rightarrow C' + AB$$

$\Rightarrow$ Three literals A, B, C'

$$\therefore A'C' + ABC + AC' + AB = C' + AB$$

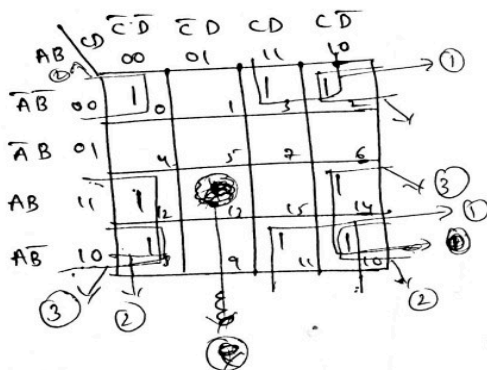


$$\begin{aligned}
 \text{Sol.} &= A'B(D'+C'D) + B(A+A'CD) \\
 &= B[A'B(D'+C'D) + A + A'CD] \\
 &= B[A'D' + A'C'D + A + A'CD] \\
 &= B[A'D' + A'D(C'+C) + A] \quad \therefore C'+C = 1 \\
 &= B[A'D' + A'D + A] \\
 &= B[A'(D'+D) + A] \quad \therefore D'+D = 1 \\
 &= B[A' + A] \quad \therefore A'+A = 1 \\
 &= B
 \end{aligned}$$

(b) Simplify the given Boolean expression using K-map $F(A,B,C,D) = \sum m(0,2,3,8,10,11,12,14)$

Sol:- Given that:-

$$F(A,B,C,D) = \sum m(0,2,3,8,10,11,12,14)$$



Group 1 - 3, 2, 11, 10.

Group 2 - 0, 2, 8, 10

Group 3 - 10, 14, 12, 8.

~~Group 4 - 0, 0, 1, 1~~

$$F = \bar{B}C + \bar{B}\bar{D} + A\bar{D}$$



9.(a) Simplify the following Boolean expressions using K-map

i) $F(x, y, z) = \sum m(2, 3, 4, 5)$ ii) $F(x, y, z) = \sum m(3, 4, 6, 7)$.

(i) Given : $F(x, y, z) = \sum m(2, 3, 4, 5)$

yz	00	01	11	10
0	0	1	1	1
1	1	1	0	0

$$F(x, y, z) = \bar{x}y + x\bar{y}$$

(ii) Given : $F(x, y, z) = \sum m(3, 4, 6, 7)$

yz	00	01	11	10
0	0	1	1	0
1	1	1	1	1

$$F(x, y, z) = yz + x\bar{z}$$

(b) Define combinational circuit and explain the procedure for designing a combinational circuit.

combinational circuit: The circuit whose output at a given instant of time depends only on the present input variables is known as combinational circuits. The combinational circuit consists of input variables, logic gates, and output variables.

The block diagram of $n \times m$ combinational circuit is shown in figure

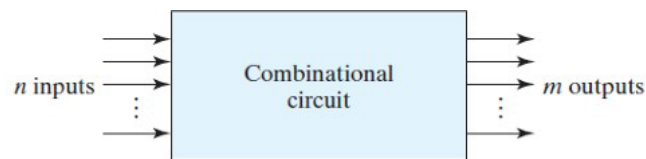


FIGURE: Block diagram of combinational circuit

DESIGN PROCEDURE:

The design of combinational circuits starts from the outline of the problem statement and ends in a logic circuit diagram or a set of Boolean functions from which the logic diagram can be easily obtained. The design procedure of the combinational circuit involves following steps:

- The problem definition.
- The determination of number of available input variables and required output variables.
- Assigning letter symbols to input and output variables.
- The derivation of truth table indicating the relationships between input and output variables.



- Obtain simplified Boolean expression for each output.
- Obtain the logic diagram.

10.(a) Explain about Binary Half Adder with truth table and logic diagram.

Half Adder: The Half adder operation needs two binary inputs: augend and addend bits; and two binary outputs: sum and carry. The truth table shown in table gives the relation between input and output variables for half adder operation.

Block diagram



Truth Table

Inputs		Output	
A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

K-Map simplification for carry and sum:

For carry

		B	
		0	1
A	0	0	0
	1	0	1
		$\bar{B}$	B

$$\text{Carry} = AB$$

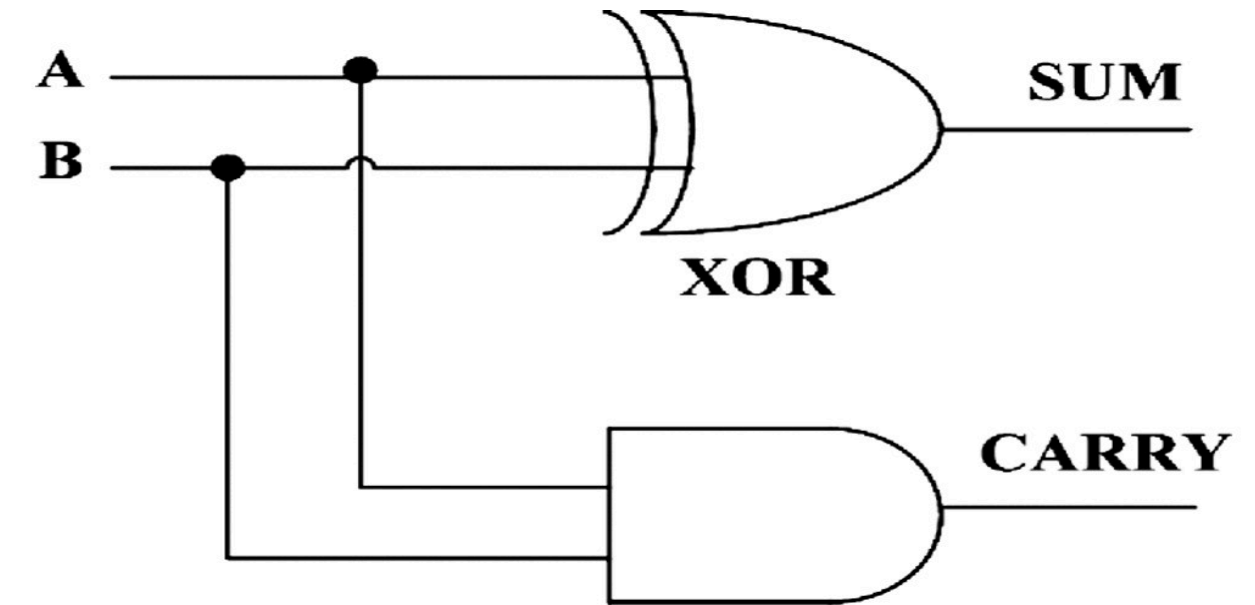
For sum

		B	
		0	1
A	0	0	1
	1	1	0
		$\bar{B}$	B

$$\begin{aligned} \text{sum} &= A'B + AB' \\ S &= A \oplus B \end{aligned}$$



Logic Diagram:

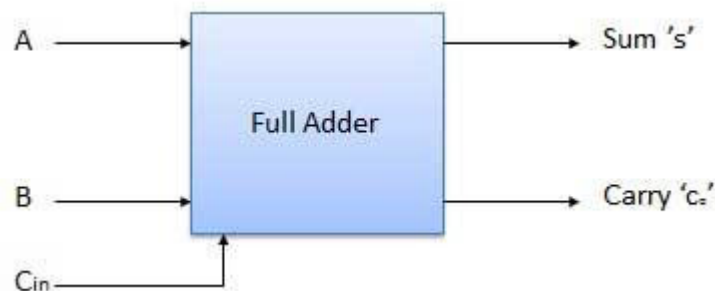


(b) Design and draw a full adder circuit.

Full Adder:

A full adder is a combinational circuit that forms the arithmetic sum of three input bits. It consists of three inputs and two outputs. Two of the input variables, denoted by A and B, represent the two significant bits to be added. The third input C_{in} represents the carry from the previous lower significant position.

Block diagram



Truth Table



Inputs			Output	
A	B	C _{in}	S	Co
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

K-Map simplification for carry and sum:-

For carry

A \ B _{cin}	00	01	11	10
0	0	0	1	0
1	0	1	1	0

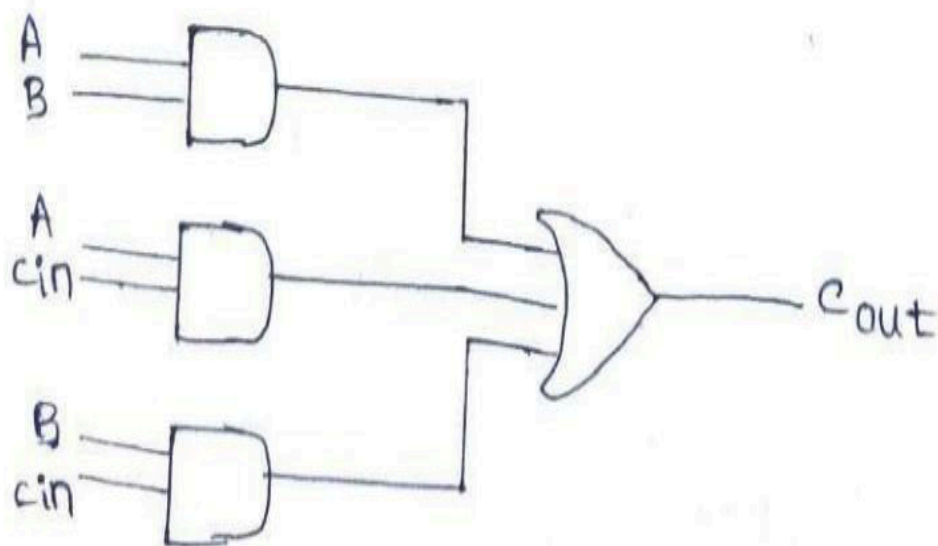
$$C_{out} = AB + AC_{in} + BC_{in}$$

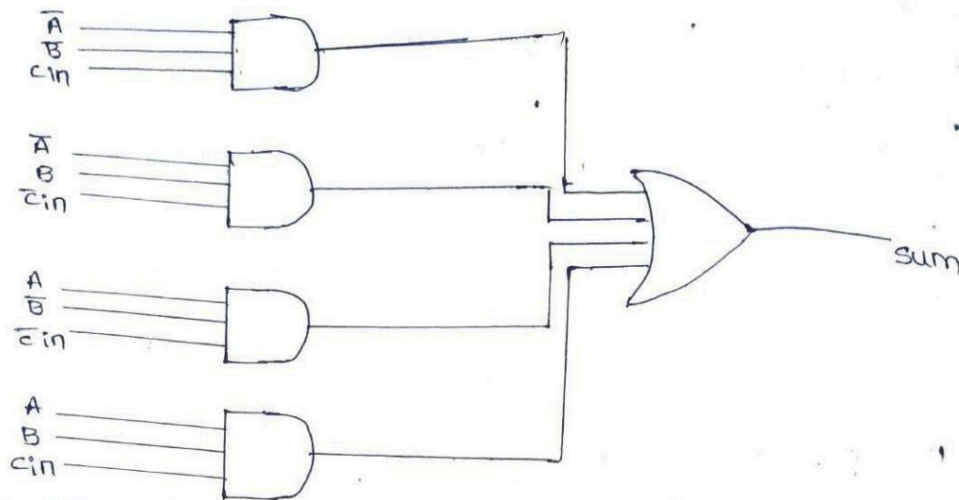
For sum

A \ B _{cin}	00	01	11	10
0	0	1	0	1
1	1	0	1	0

$$Sum = \bar{A}\bar{B}C_{in} + \bar{A}B\bar{C}_{in} + A\bar{B}\bar{C}_{in} + ABC_{in}$$

logic diagram:-



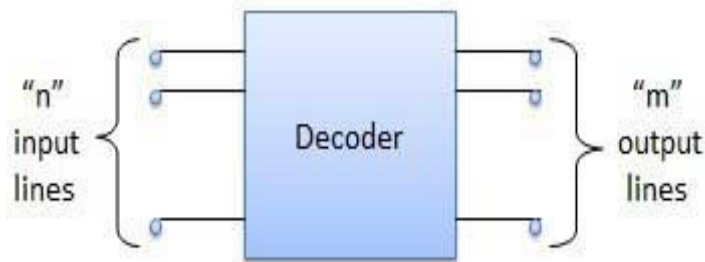


11.(a) Define Decoder and explain in detail about a 2-to-4-line binary

decoder. Decoder:

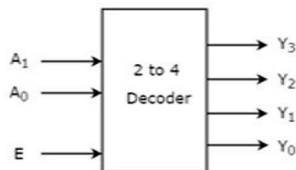
A decoder is a combinational circuit. It has n input and to a maximum $m = 2^n$ outputs. Decoder is identical to a Demultiplexer without any data input. It performs operations which are exactly opposite to those of an encoder.

Block diagram



2 to 4 Decoder

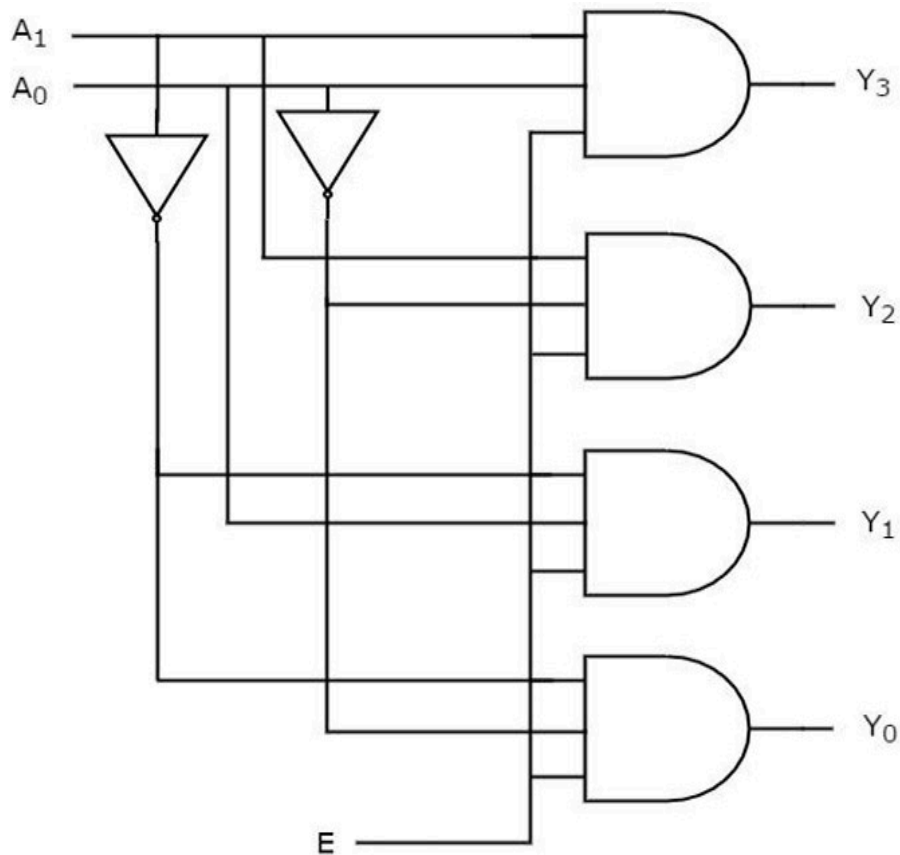
Let 2 to 4 Decoder has two inputs A_1 & A_0 and four outputs Y_3, Y_2, Y_1 & Y_0 .



Enabl e	Inputs		Outputs			
E	A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	x	x	0	0	0	0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0

$$\begin{aligned}
 Y_3 &= E \cdot A_1 \cdot A_0 \\
 Y_2 &= E \cdot A_1 \cdot A_0' \\
 Y_1 &= E \cdot A_1' \cdot A_0 \\
 Y_0 &= E \cdot A_1' \cdot A_0'
 \end{aligned}$$



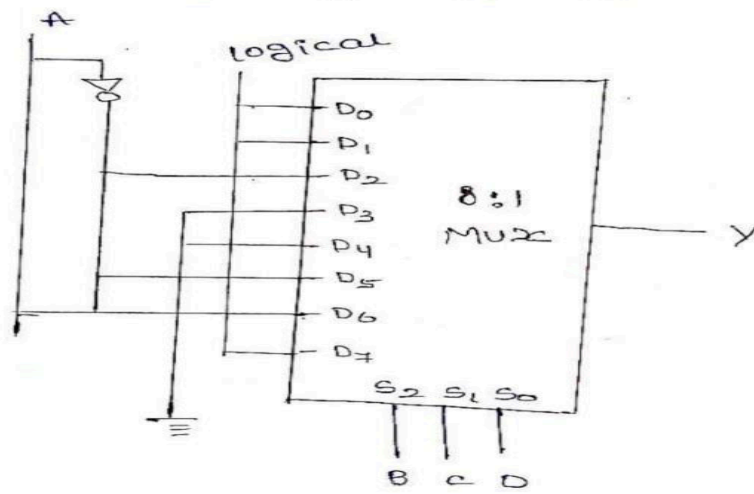


(b) Design and implement the following Boolean function by 8:1 Multiplexer.
 $(A,B,C,D)=\Sigma m(0,1,2,5,7,8,9,14,15)$.

$$F(A,B,C,D) = \Sigma m(0,1,2,5,7,8,9,14,15)$$

	D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7
$\bar{A}$	0	1	2	3	4	5	6	7
A	8	9	10	11	12	13	14	15
	1	1	$\bar{A}$	0	0	$\bar{A}$	A	1







SIDDHARTH INSTITUTE OF ENGINEERING & TECHNOLOGY (AUTONOMOUS)

(Approved by AICTE, New Delhi & Affiliated to JNTUA, Ananthapuramu)

(Accredited by NBA for Civil, EEE, Mech., ECE & CSE)

(Accredited by NAAC with 'A+' Grade)

Puttur -517583, Tirupati District, A.P. (India)

UNIT- II PART-

A (2 MARKS)

1(a) Define a sequential circuit and draw its block diagram.

Sequential circuit: A sequential logic circuit is a digital circuit whose output depends not only on the present input but also on the past inputs. It has a memory element like a flip-flop or latch which stores the past sequence of inputs. The working of a sequential circuit involves updating its memory elements in response to changes in the input. Unlike combinational circuits, sequential circuits can process streams of input data over a period of time.

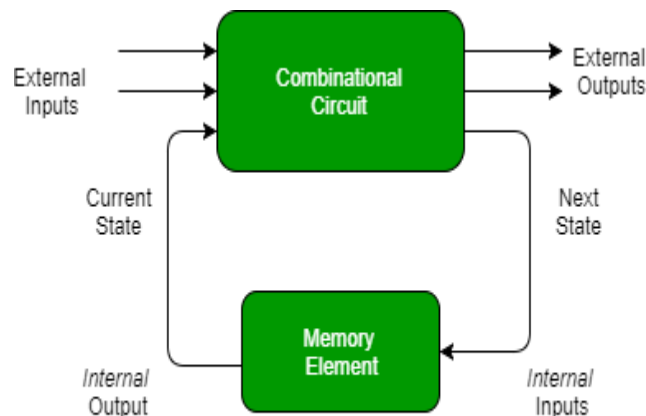


Figure: Sequential Circuit

(b) What is Flipflop and different types of Flipflop?

Flip-flops: The flip-flop is a circuit that maintains a state until directed by input to change the state. A basic flip-flop can be constructed using four-NAND or four-NOR gates. The flip-flop is popularly known as the basic digital memory circuit. Its two states are logic 1(High) and logic 0(low) states. A flip-flop is a sequential circuit that consists of a single binary state of information or data.

Types of Flip-Flops: There are Four types of Flip-Flops

- S-R Flip-Flop



- J-K Flip-Flop
- D Flip-Flop
- T Flip-Flop

(c) List the types of Buses.

There are three types of bus lines: Data bus, Address bus, and Control bus. Communication over each bus line is performed in cooperation with another. The data bus is a signal line for exchanging the data between the CPU and the memory, and between the CPU and I/O, and handles the data stored in the specified location.

(d) Sketch the basic functional units of computer.

A computer in its simplest form comprises of five functional units namely input unit, output unit, memory unit, arithmetic & logic unit and control unit.

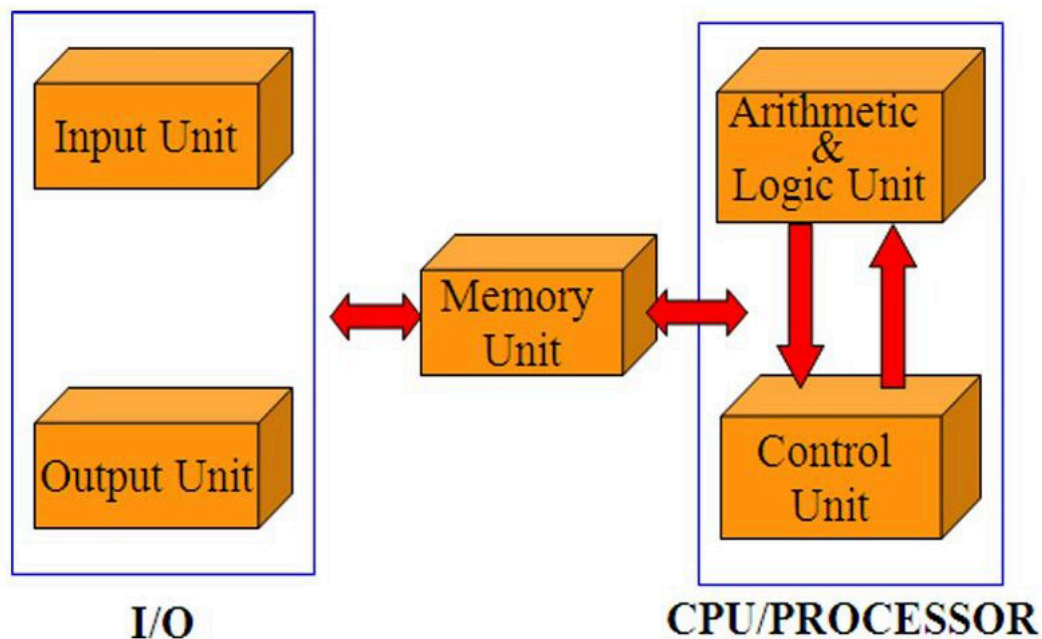


Figure 1: Functional units of a computer

(e) List the Classification of Computer Generations.

The five generations of computers are classified by the type of hardware used in each generation:

- **First generation:** Vacuum tubes were used from the 1940s–1950s.
- **Second generation:** Transistors were used from the 1950s–1960s.
- **Third generation:** Integrated circuits were used from the 1960s–1970s.
- **Fourth generation:** Microprocessors have been used since the 1970s.
- **Fifth generation:** Artificial intelligence is the focus of current and future computers.



PART-B (10 MARKS)

2(a) Differentiate between Combinational & Sequential circuits.

Combinational Circuits	Sequential Circuits
The outputs of the combinational circuit depend only on the present inputs.	The outputs of the sequential circuits depend on both present inputs and present state (previous output).
The feedback path is not present in the combinational circuit.	The feedback path is present in the sequential circuits.
In combinational circuits, memory elements are not required.	In the sequential circuit, memory elements play an important role and are mandatory.
The clock signal is not required for combinational circuits.	The clock signal is required for sequential circuits.
The combinational circuit is simple to design.	Sequential circuit is complex to design.

(b) Distinguish between latches and flipflops.

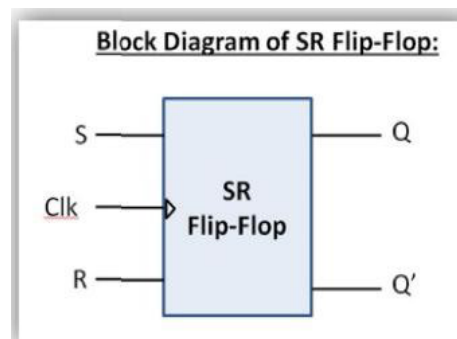
Latch	Flip-flop
Latch follows a level triggering approach	Flip-flop utilizes an edge triggering approach.
The clock signal is absent.	The clock signal is present.
You can design it using Logic gates.	You can design it using Latches along with a clock.
It has a comparatively fast operating speed.	It has a slow operating speed.
A user cannot classify the Latch this way.	You can classify a flip-flop into a synchronous or asynchronous flip-flop.
It requires comparatively less power.	It requires more power.
J-K, S-R, D, and T Latches.	J-K, S-R, D, and T Flip-flops.

3(a) Explain the working principle of SR and JK flip-flops

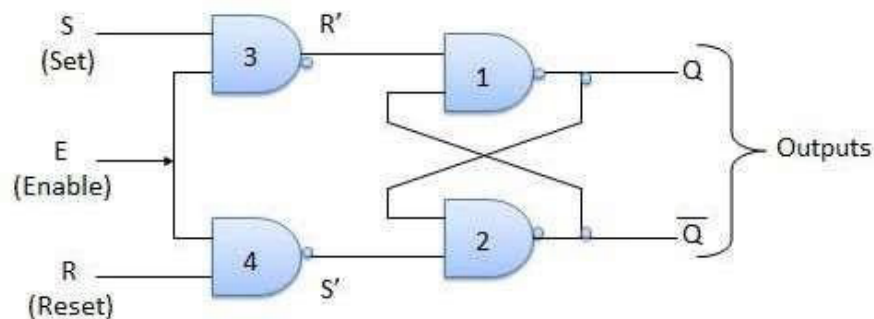
S-R FlipFlop: This is the simplest flip-flop circuit. It has a set input (S) and a reset input (R). When in this circuit when S is set as active, the output Q would be high and the Q' will be low. If R is set to active then the output Q is low and the Q' is high.



Block Diagram



Circuit Diagram



Truth Table

Inputs			Outputs		Comments
E	S	R	Q_{n+1}	$\overline{Q}_{n+1}$	
1	0	0	Q_n	$\overline{Q}_n$	No change
1	0	1	0	1	Rset
1	1	0	1	0	Set
1	1	1	x	x	Indeterminate

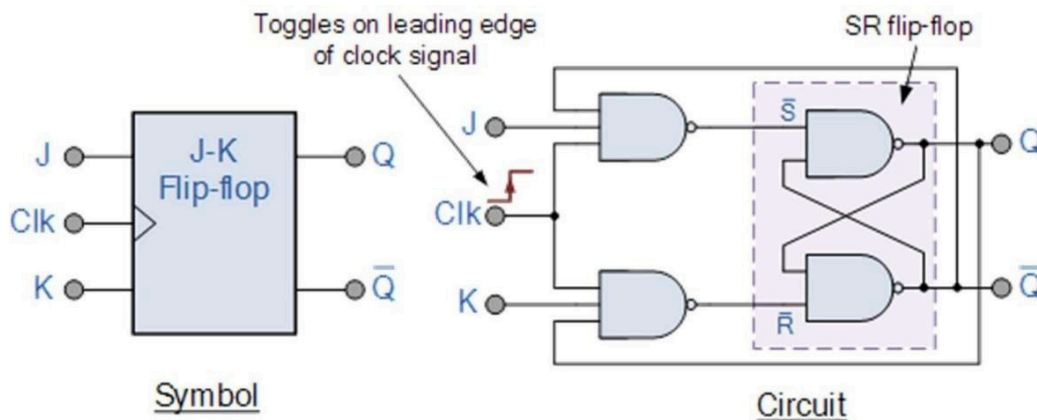
Operation:

- If $S=0$, $R=0$ then the output of NAND gate are forced to become 1. Hence R' and S' both will be equal to 1. There will be no change in the state of outputs.
- If $S=0$, $R=1$ then the output of NAND gate the is $R'=1$ and the output of NAND gate is $S'=0$. Hence $Q_{n+1}=0$ and $\overline{Q}_{n+1}=1$. This is Reset condition.
- If $S=1$, $R=0$ then the output of NAND gate the is $R'=0$ and the output of NAND gate is $S'=1$. Hence $Q_{n+1}=1$ and $\overline{Q}_{n+1}=0$. This is set condition.



- If $S=1$, $R=1$ the output of NAND gate that is $R'=0$, $S'=0$ both are 0. Hence the race condition will occur in the basic NAND gate latch.

JK Flipflop: JK Flipflop is a modification of SR Flipflop. The J input is similar to the set input of a SR flipflop and the K input is similar to the Reset input of SR Flipflop. The condition $J=K=1$ which is not allowed in SR Flipflop is interpreted as a toggle condition. The JK Flipflop has two data inputs and two data outputs, one clock signal.



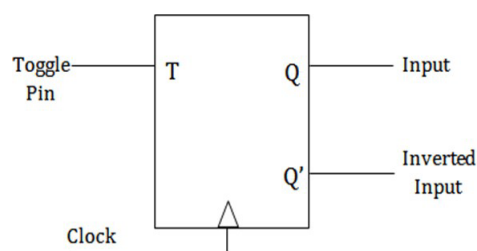
Operation:

- When $J=0$ and $K=0$ then Q returns its previous state and it holds the current state.
- When $J=0$ and $K=1$ then the flipflop will be in reset that is $Q_n=0$ and $Q_n'=1$.
- When $J=1$ and $K=0$ then the flipflop will be in set that is $Q_n=1$ and $Q_n'=0$.
- When $J=1$ and $K=1$ then the flipflop will be in toggle state. This means that the output will be complement to the previous state.

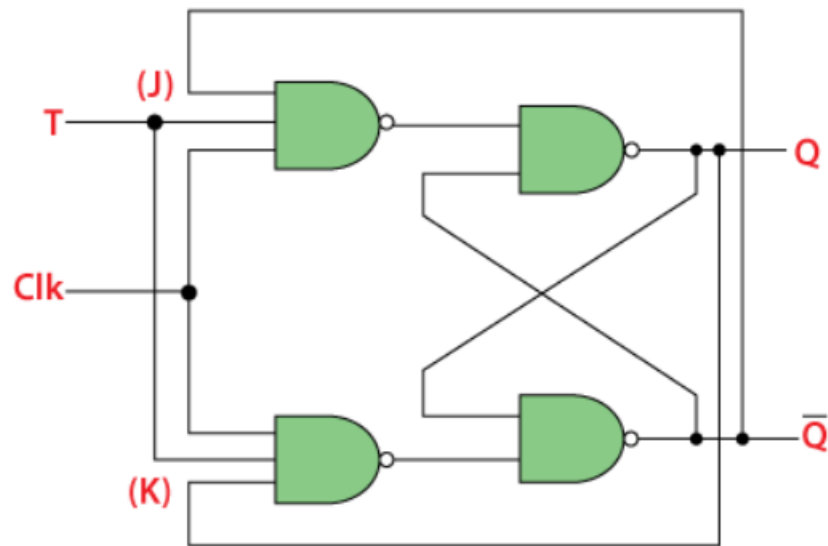
(b) Explain the working principle of T and D flip-flops.

T-FlipFlop: A T-Flipflop is a simple version of JK Flipflop. The T-flipflop is obtained by connecting the JK flipflop inputs together and the flipflop has one input terminal and clock input. These flipflops are said to be T-flipflops because of their ability to toggle input state. T-flipflops are mostly used in counters.

Block Diagram



Circuit Diagram



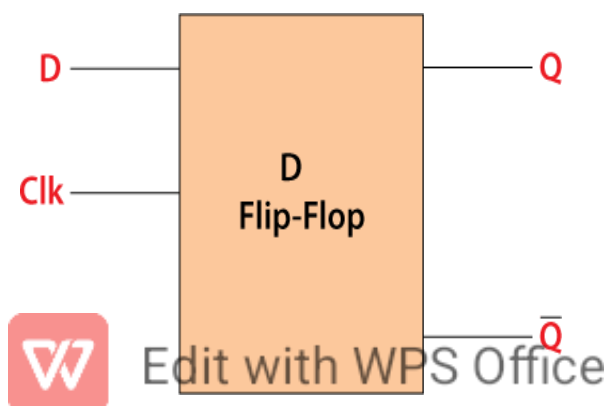
T-Flipflop Truth Tables:

T	Q	Q(t+1)
0	0	0
1	0	1
0	1	1
1	1	0

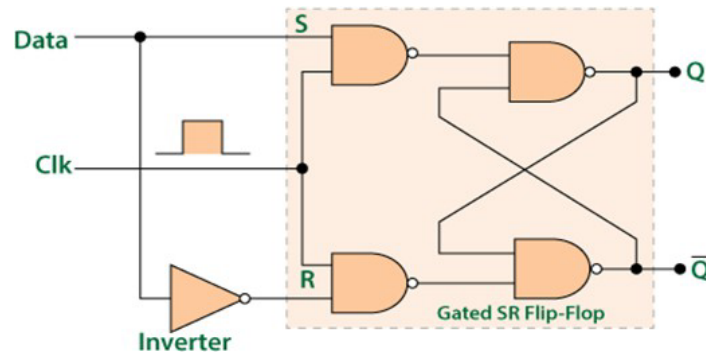
CLK	T	Q(n+1)	State
	0	Q	NO CHANGE
	1	Q'	TOGGLE

D-FlipFlop: In a D-Flipflop the output can only be changed at positive or negative clock and when the inputs changed at other times the output will remain unaffected. The D-flipflops are generally used for shift registers and counters.

Block Diagram



Circuit Diagram



D-Flipflop Truth Tables:

D Flip Flop		
Input	Output	
D	Q	$Q^{\wedge}$
0	0	1
1	1	0

Working:

In D flip flop, the single input "D" is referred to as the "Data" input. When the data input is set to 1, the flip flop would be set, and when it is set to 0, the flip flop would change and become reset.

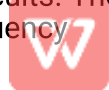
4(a) What is a counter? List the applications of counters.

Counter: Counter is a sequential circuit. A digital circuit which is used for a counting pulse is known counter. Counter is the widest application of flip-flops. It is a group of flip-flops with a clock signal applied. Counters are specially designed synchronous sequential circuits, in which, the state of the counter is equal to the count held in the circuit by the flip flops.

Applications of counters:

Counter found their applications in many digital electronic devices. Some of their applications are listed below.

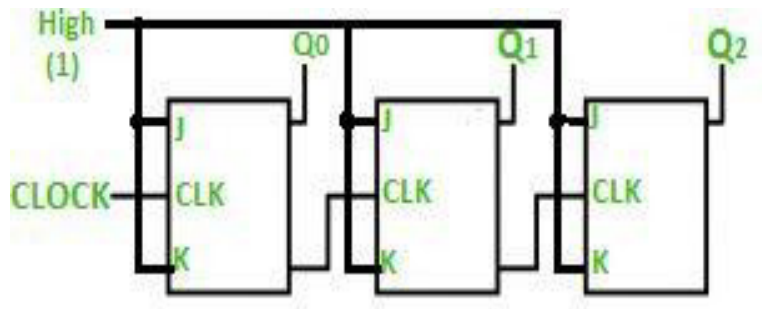
- o Frequency counters
- o Digital clocks
- o Analog to digital convertors.
- o With some changes in their design, counters can be used as frequency divider circuits. The frequency divider circuit is that which divides the input frequency



exactly by '2'.

- o In time measurement means calculating time in timers such as electronic devices like ovens and washing machines.
- o We can design digital triangular wave generator by using counters.

(b) Explain in detail about 3-bit ripple Up-counter using suitable diagram.



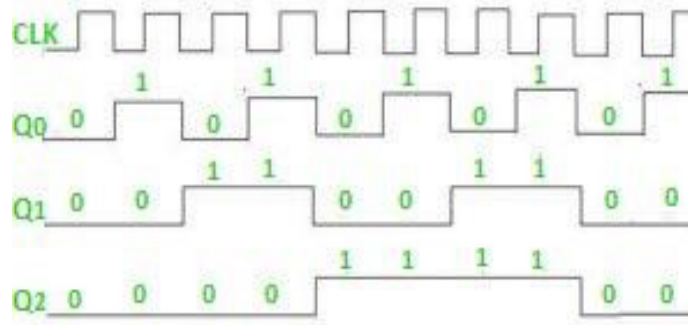
In the circuit shown in the above figure, Q0(LSB) will toggle for every clock pulse because JK flip-flop works in toggle mode when both J and K are applied 1, 1, or high input. The following counter will toggle when the previous one changes from 1 to 0. When counting up, for the n-bit counter the count sequence goes from 000, 001, 010, ... 110, 111, 000, 001, ... etc.

Truth Table is as follows:

Counter State	Q ₂	Q ₁	Q ₀
0	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0
5	1	0	1
6	1	1	0
7	1	1	1

Timing diagram – Let us assume that the clock is negative edge triggered so the above counter will act as an up counter because the clock is negative edge triggered and output is taken from Q.





5 Define a Register. Explain in detail about various Shift Registers.

Register: A Register is a collection of flip flops. A flip flop is used to store single bit digital data. For storing a large number of bits, the storage capacity is increased by grouping more than one flip flops. If we want to store an n-bit word, we have to use an n-bit register containing n number of flip flops.

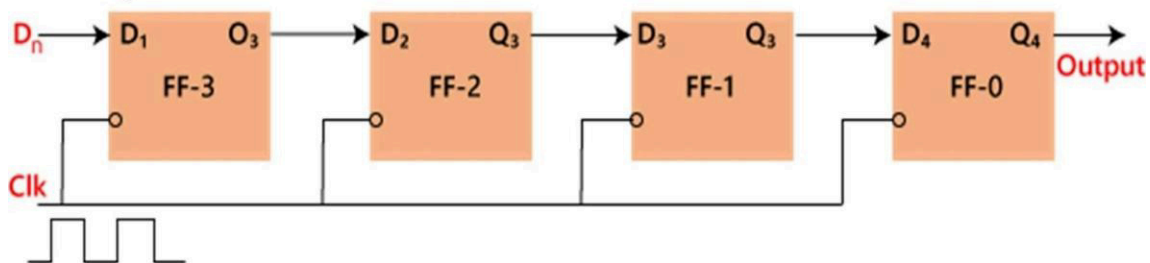
Shift Register: A group of flip flops which is used to store multiple bits of data and the data is moved from one flip flop to another is known as Shift Register

The shift register is classified into the following types:

- Serial In Serial Out
- Serial In Parallel Out
- Parallel In Serial Out
- Parallel In Parallel Out

Serial IN Serial OUT: In "Serial Input Serial Output", the data is shifted "IN" or "OUT" serially. In SISO, a single bit is shifted at a time in either right or left direction under clock control. The block diagram of the "Serial IN Serial OUT" is given below.

Block Diagram:

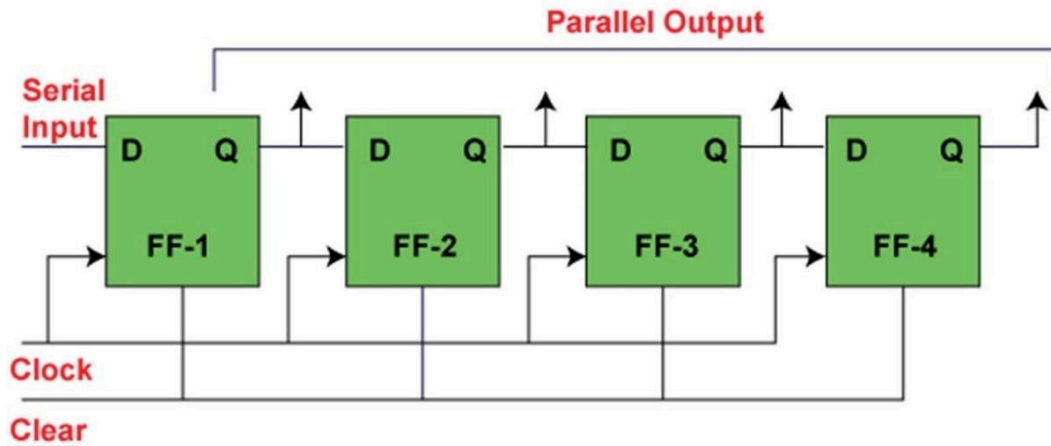


Serial IN Parallel OUT: In the "Serial IN Parallel OUT" shift register, the data is passed serially to the flip flop, and outputs are fetched in a parallel way. Below is the block diagram of the 4-bit **serial in the parallel-out** shift register. The circuit having four D flip-flops contains a clear and clock signal to reset these four flip flops. In **SIPO**, the input of the second flip flop is the output of



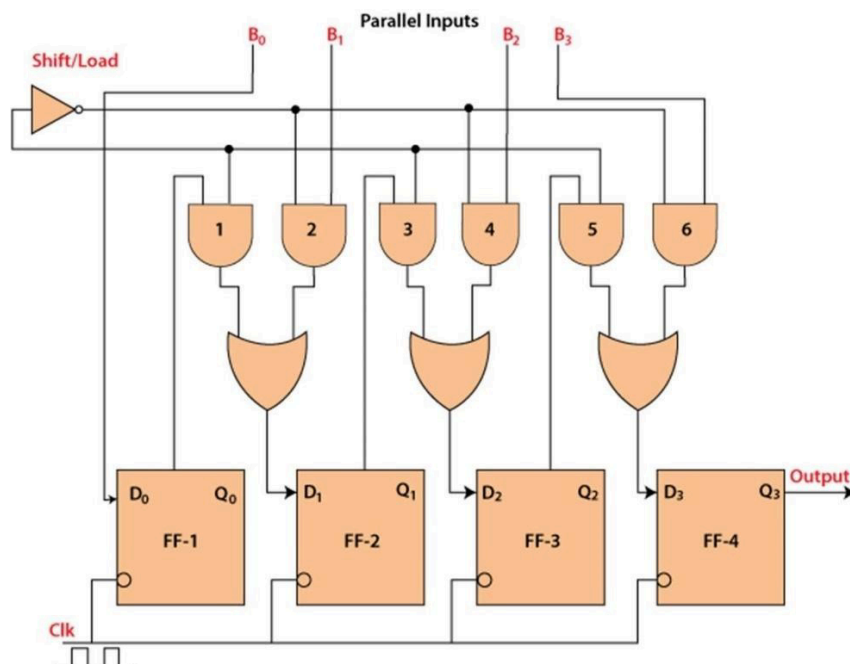
the first flip flop, and so on. The same clock signal is applied to each flip flop since the flip flops synchronize each other. The parallel outputs are used for communication.

Block Diagram:



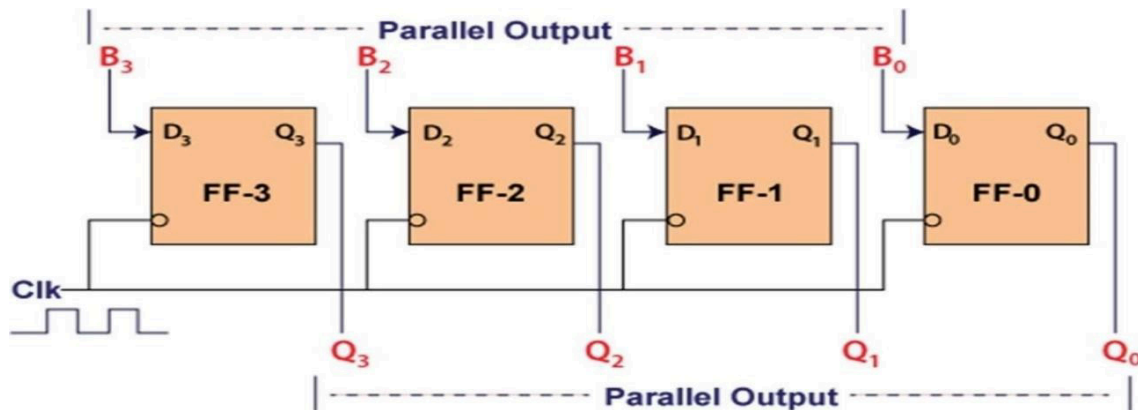
Parallel IN Serial OUT: In the "Parallel IN Serial OUT" register, the data is entered in a parallel way, and the outcome comes serially. A four-bit "Parallel IN Serial OUT" register is designed below. The input of the flip flop is the output of the previous Flip Flop. The input and outputs are connected through the combinational circuit. Through this combinational circuit, the binary input B_0, B_1, B_2, B_3 are passed.

Block Diagram:



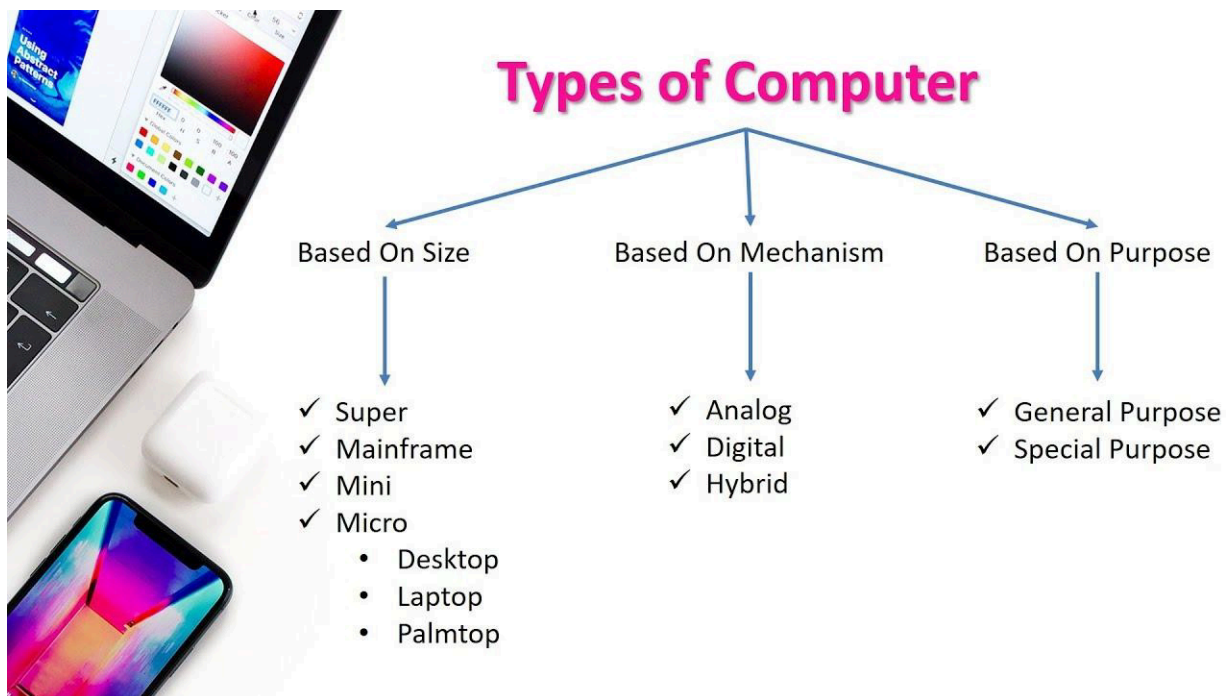
Parallel IN Parallel OUT: In "Parallel IN Parallel OUT", the inputs and the outputs come in a parallel way in the register. The inputs A0, A1, A2, and A3, are directly passed to the data inputs D0, D1, D2, and D3 of the respective flip flop. The bits of the binary input are loaded to the flip flops when the negative clock edge is applied. The clock pulse is required for loading all the bits. At the output side, the loaded bits appear.

Block Diagram:



6(a) List the different types of a computer.

Computer Types: The computer types mainly classified into three types based on size, mechanism and purpose.



Super Computer: Super Computer is invented by Roger Cray in 1970s. The super computer is the fastest and largest computer among all types of computers. It performs trillions of instructions in few seconds. The size of the super computer is around one room capacity. It is very expensive. Examples: CDC, Cyber, IBM and TITAN

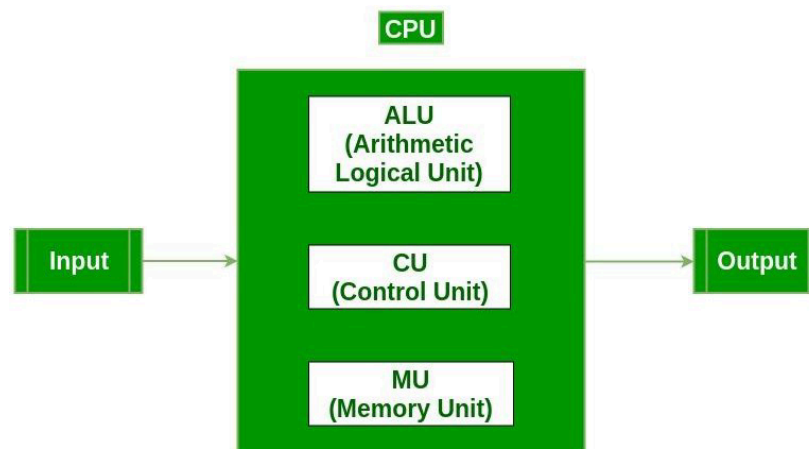
Mainframe Computer: In mainframe computers thousands and hundreds of users can process the input data at a time. The mainframe computers can perform millions of instructions within a few seconds. The mainframe computers are also expensive. Examples: Banking and Telecom sectors.

Mini Computer: Mini computers are also called as medium size computers. It performs 4-200 instructions within a few seconds. The mini computers are developed in 1980s. Examples: Accounting for billing purpose in institutions.

Micro Computer: Micro computers are also known as personal computers and are designed to be used individually. Examples: Laptops, Desktops, Tablets and Smart Phones.

(b) Explain the functional units in the computer.

Functional Unit:



Input Unit: An input device is usually a keyboard and mouse through these devices. We can give input data/instructions to the memory. The common input device is keyboard which accepts letters, numbers and communicates from the user.

Memory Unit: Memory is used to store the programs and data. There are two classes of storage called primary and secondary memory.

Primary Memory: It is the fast memory that operates at electronic speed, programs must stay in memory while they are being executed. The memory contains a large number of semiconductor storage cells. Each cell is capable of storing one bit of information.



Secondary Memory: Secondary memory is used when large amount of data and many programs have to be store particularly for information that is asking in frequently.

Arithmetic and logic unit: The Arithmetic section performs Arithmetic operations such as Addition, Subtraction, Multiplication and Division. The Logic section performs logical operations such as comparing two numbers to see the number is equal to less than or greater than the other.

Control unit: All activities inside the computer are directed and controlled by the control unit. A control unit works by receiving input information from memory to which it converts into control signals, these signals are sent to the central processing unit for doing operations.

Output unit: In computing input/output refers to the communication between an information processing system and the outside world. Inputs are the signals or data sent to the system and outputs are the signals or data sent by the system to the outside. Examples: Speakers, Monitor Screen, Printer

7(a) Differentiate between I/O unit and memory unit.

S.No	Isolated I/O	Memory Mapped I/O
1	Isolated I/O uses separate memory space.	Memory mapped I/O uses memory from the main memory.
2	Limited instructions can be used. Those are IN, OUT, INS, OUTS	Any instruction which references to memory can be used.
3	The addresses for Isolated I/O devices are called ports.	Memory mapped I/O devices are treated as memory locations on the memory map.
4	IORC & IOWC signals expands the circuitry.	IORC & IOWC signals has no functions in this case which reduces the circuitry.
5	Efficient I/O operations due to using separate bus	Inefficient I/O operations due to using single bus for data and addressing
6	Comparatively larger in size	Smaller in size
7	Uses complex internal logic	Common internal logic for memory and I/O devices
8	Slower operations	Faster operations

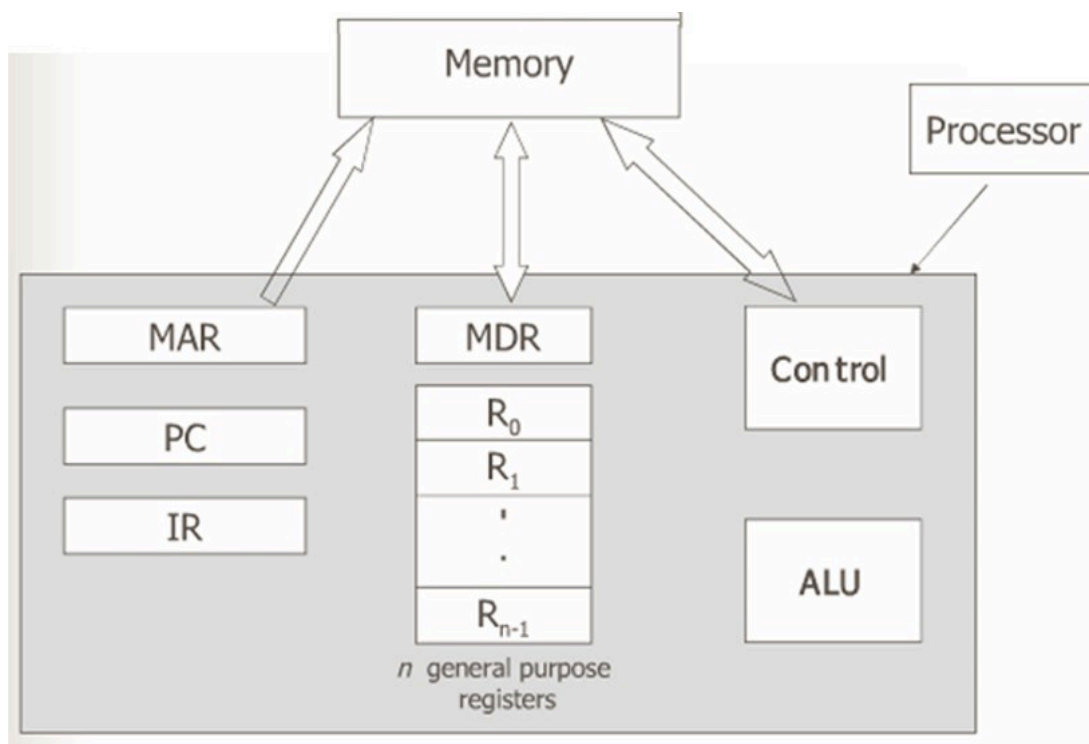


(b) Differentiate between control unit and ALU.

S.No	Control Unit	Arithmetic Logic Unit
1	It directs the system and executes the instructions entered by the user. It directs and coordinates most of the operations in the computer.	To perform arithmetic operations, comparison operations, and other operations. It performs all the tasks to complete all those instructions.
2	After the completion of the operation, the data gets deleted and it is not accessible.	Data is accessible at any time.
3	It is responsible to direct the system and execute the instructions.	It is responsible to perform all the tasks to complete the CU instruction.
4	Equal-to conditions, Less-than condition, and greater than the state.	Loading of data and instructions that exist in the secondary memory to the main memory.
5	No dependency on ALU.	ALU depends on CU for most of the time.

8 Describe the Basic Operational Concepts of computer with neat diagram.

Basic Operational Concepts:



- The program counter a list of instructions is stored in the memory.



- Individual instructions are brought from the memory into the processor which executes the specified operations.
- Data to be used as operands also stored in the memory

ADD R1,R2,R3

In these instructions ADD is the operation perform on operands R1, R2 and are placed the result stored in R3.

Instruction Register (IR): The instruction register holds the instructions that is currently being executed. Its output is available to the control circuit which generates the timing signals that control various processing elements involved in executing the instruction.

Program Counter (PC): The program counter is another specialized register. It keeps track on execution of program. It contains memory address of the next instruction to be fetched and executed.

Memory Address Register & Memory Data Register (MAR & MDR): These two registers maintain communication with memory. The Memory Address Register holds the address of the particular instruction. The Memory Data Register contains the data to be written into readout of the address location.

9(a) Give the Structure of BUS Interface with various devices in computer.

Bus Structure: A group of lines that connects several devices of a computer is called as Bus. The following are different types of Buses:

- Address Bus
- Data Bus
- Control Bus

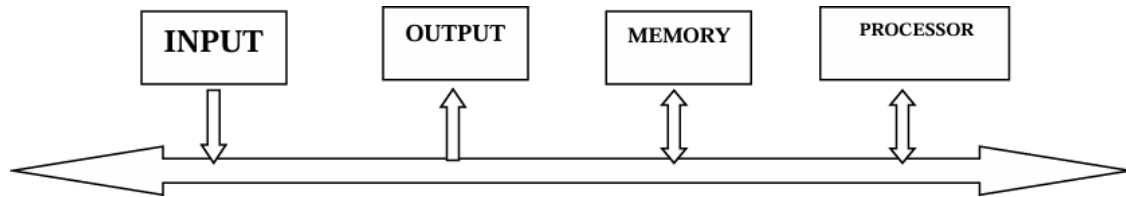
Address Bus: The Address bus is the set of lines that carry address information to the memory where the data is to be transferred. It is a unidirectional bus, the address bus consists of 16, 20, 24 or more signal lines. On these lines the CPU sends address of the memory location.

Data Bus: The data from one component to another component is connected with help of data bus. The data bus consists of 8, 16, 32 or more parallel signal lines. The data bus lines are Bi-directional.

Control Bus: The control bus carries control and timing information.



Single Bus Structure: A group of lines services as connecting path for several devices of a computer is called as Bus. In single bus structure all units are connected to this bus because the bus can be used only 2 units can be connect actively to use the bus. Bus control lines are used to send multiple request to use the bus.



(b) Explain briefly about the software of a computer.

Software: A total computer system includes both software and Hardware.

- Hardware consists of physical components and all associated equipment.
- Software refers to the collection programs that are written for the computer and writing a program for a computer consists of specifying, directly or indirectly a sequence of machine instructions.
- The computer software consists of the instructions and data that the computer manipulates to perform various data processing tasks.

Types: 1. Application software, 2. System software

System Software: System software is used to run application software. System software is a collection of programs that are executed as needed to perform functions such as

1. Receiving and interpreting user commands.

2. Entering and editing application programs and sorting them as files in secondary storage devices. (Editor)

Application software: Application software allows end users to accomplish one or more specific (not directly computer development related) tasks. Its usually written in high level languages, such as c , c++ , java.

Typical applications include:

- Word processing
- spreadsheet
- computer games
- databases
- industrial automation
- medical software

10(a) Explain briefly about the performance of a computer.

Performance: The most important measure of the performance of a computer is how quickly it can compute programs. At the start of execution, all program instructions and the required data are stored in the memory as shown below.



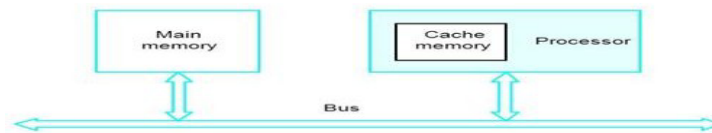


Figure 1.5. The processor cache.

Computer performance is often described in terms of clock speed (usually in MHz or GHz). This refers to the cycles per second of the main clock of the CPU. Performance of a computer depends on the following factors.

Basic performance equation: The Performance Equation is a term used in computer science. It refers to the calculation of the performance or speed of a central processing unit (CPU).

$$T = (N \cdot S) / R$$

Where 'T' is the processor time

'N' is the total number of steps required to complete program execution.

'S' is the average number of basic steps each instruction execution

requires, 'R' is the clock rate [In cycles per second]

(b) Explain briefly about the multi processors and multi computers of a

computer. Multi Processors and Multi Computers:

- A multi-processor system has more than one processor (CPU), with common memory shared between processors.
- A multi-processor may be on a common bus (close coupled) or distributed on a network loosely coupled.
- A multi-computer system has more than one processor with each processor having local memory.

Multi-processing systems: Generally accepted definition of a multiprocessing/ multi computing system are

- Multiple processors, each with its own CPU and Memory
- Interconnection hardware
- Processors fail independently
- There exists a shared state
- Appears to users as single system

11(a) List the different types of computer

generations. Computer Generations:

The history of computer development is often referred to in reference to the different generations of computing devices. Each generation of computer is characterized by a major technological development that fundamentally changed the way computers operate, resulting in increasingly smaller, cheaper, more powerful and more efficient and reliable devices.

a) First Generation (1940-1956): Vacuum Tubes: The first computers used vacuum tubes for circuitry and magnetic drums for memory, and were often enormous, taking up

entire rooms. They were very expensive to operate and in addition to using a great deal of electricity, generated a lot of heat, which was often the cause of malfunctions.

Example: The UNIVAC and ENIAC computers are examples of first-generation computing devices.

b) Second Generation (1956-1963): Transistors: Transistors replaced vacuum tubes and ushered in the second generation of computers. The transistor was invented in 1947 but did not see widespread use in computers until the late 1950s. The transistor was far superior to the vacuum tube, allowing computers to become smaller, faster, cheaper, more energy-efficient and more reliable than their first-generation predecessors. High-level programming languages were also being developed at this time, such as early versions of COBOL and FORTRAN.

c) Third Generation (1964-1971): Integrated Circuits: The development of the integrated circuit was the hallmark of the third generation of computers. Transistors were miniaturized and placed on silicon chips, called semiconductors, which drastically increased the speed and efficiency of computers.

Example: punched cards and printouts

d) Fourth Generation (1971-Present): Microprocessors: The microprocessor brought the fourth generation of computers, as thousands of integrated circuits were built onto a single silicon chip. What in the first generation filled an entire room could now fit in the palm of the hand. The Intel 4004 chip, developed in 1971, located all the components of the computer— from the central processing unit and memory to input/output controls—on a single chip. In 1981 IBM introduced its first computer for the home user, and in 1984 Apple introduced the Macintosh.

e) Fifth Generation (Present and Beyond): Artificial Intelligence): Fifth generation computing devices, based on artificial intelligence, are still in development, though there are some applications, such as voice recognition, that are being used today. The use of parallel processing and superconductors is helping to make artificial intelligence a reality. The goal of fifth-generation computing is to develop devices that respond to natural language input and are capable of learning and self-organization.

(b) Explain briefly about the Von- Neumann Architecture of a computer.

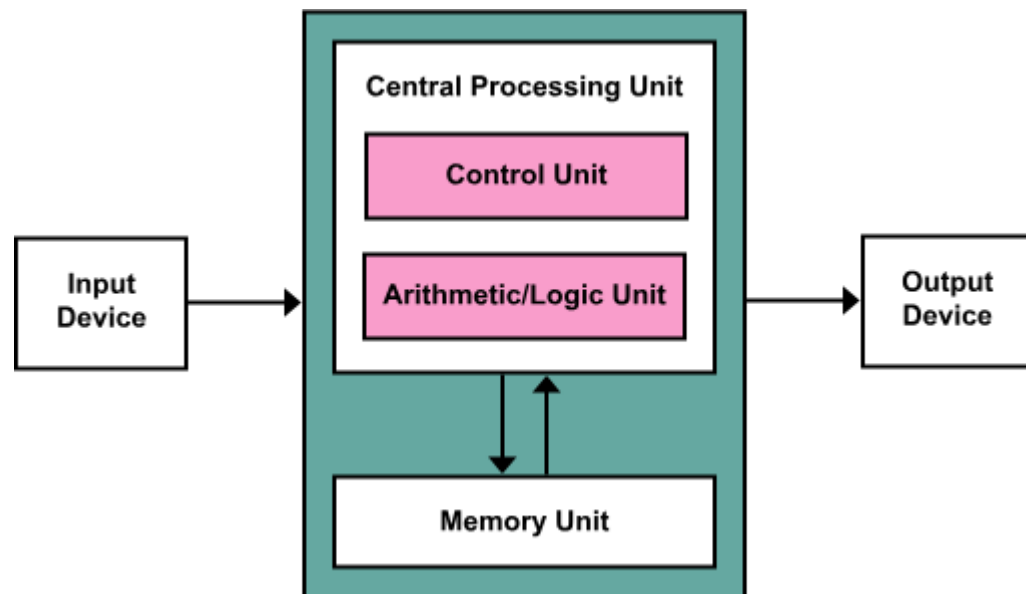
Von-Neumann Architecture:

- A major defining point in the history of computing notes the realization in 1944 to 1945 that data and instructions to manipulate data were logically the same and could be stored in the same place.
- The computer design built upon the principle which became known as the Von-Neumann architecture is still the basis for computer today.

Von-Neumann Model: The Von-Neumann Architecture or Von-Neumann model is also known as “Princeton Architecture”. This architecture was published by the Mathematician John Von Neumann in

1945. Von Neumann architecture is the design upon which many general-purpose computers are based. This architecture implemented the stored program concept in which the data and instructions are stored in the same memory. This architecture consists of a CPU (ALU,

Registers, Control Unit), Memory and I/O unit.



- A Processing unit that contains an arithmetic logic unit and processor registers.
- A Control unit that contains an instruction register and program counter.
- Memory means that stores data and instructions.
- External/ mass storage
- Input and Output Mechanism.

The term Von-Neumann Architecture has evolved to mean any stored program computer in which an instruction fetch and data operation cannot occur at the same time because they share a common memory.





SIDDHARTH INSTITUTE OF ENGINEERING & TECHNOLOGY

(AUTONOMOUS)

(Approved by AICTE, New Delhi & Affiliated to JNTUA, Ananthapuramu)

(Accredited by NBA for Civil, EEE, Mech., ECE & CSE)

(Accredited by NAAC with 'A+' Grade)

Puttur -517583, Tirupati District, A.P. (India)

UNIT- III PART

-A (2 MARKS)

1(a) Represent -7 in signed magnitude, 1s complement and 2s complement.

To represent -7 in signed magnitude, 1's complement, and 2's complement, we first need to express the number 7 in binary and then apply the necessary transformations for each representation.

Let's assume we're using an 8-bit representation.

1. Signed Magnitude Representation:

- In signed magnitude, the most significant bit (MSB) is the sign bit, where:
 - 0 means positive,
 - 1 means negative.
- First, represent the absolute value of 7 in binary.
 - $7_{10} = 00000111_2$ (using 8 bits).
- Since the number is negative, the sign bit will be 1, and the rest of the bits will represent the magnitude.
 - Signed Magnitude: 10000111

2. 1's Complement Representation:

- In 1's complement, negative numbers are obtained by flipping all bits of the corresponding positive number.
- First, represent 7 in binary:
 - $7_{10} = 00000111_2$.
- Then, flip all the bits to get the 1's complement of 7, which gives:
 - 1's Complement: 11111000



Edit with WPS Office

3. 2's Complement Representation:

- In 2's complement, negative numbers are obtained by first taking the 1's complement and then adding 1 to the result.
- Starting with the 1's complement of 7, which is 11111000, add 1:
 - $11111000 + 1 = 11111001$
- 2's Complement: 11111001

(b) What is the advantage of using Booth algorithm?

- 1) It handles both positive and negative multiplier uniformly.
- 2) It achieves efficiency in the number of additions required when the multiplier has a few large blocks of 1's.
- 3) The speed gained by skipping 1's depends on the data.

(c) What is floating point numbers?

In some cases, the binary point is variable and is automatically adjusted as computation proceeds. In such case, the binary point is said to float and the numbers are called floating point numbers.

(d) What are the basic operations to execute a complete instruction?

To execute a complete instruction in a computer's processor (CPU), several basic operations are typically involved. These operations can be grouped into the following stages:

1. Fetch:

- Operation: The instruction is retrieved from memory (usually from RAM) using the program counter (PC), which contains the address of the next instruction to be fetched.
- Explanation: The PC is incremented after fetching the instruction to point to the next one.

2. Decode:

- Operation: The fetched instruction is decoded by the control unit to understand what operation is required. This involves parsing the opcode (operation code) and operands (data or addresses) of the instruction.
- Explanation: The instruction is broken down into its components, like the operation type, source, and destination operands.

3. Execute:



- o Operation: The actual computation or operation specified by the instruction is performed. This could involve arithmetic operations, data transfer, or branching.
- o Explanation: If the instruction involves an arithmetic operation, the Arithmetic Logic Unit (ALU) carries it out. For memory access instructions, data might be loaded or stored.

(e) What is the need of multiple organization?

The need of multiple organization is to manage and utilize computer resources efficiently enabling

- Efficient memory management and hierarchy
- Instruction-level parallelism and pipelining for improved processing.

PART-B (10 MARKS)

2(a) Explain the Flow chart for Addition and

Subtraction. Addition and Subtraction with Signed

–Magnitude Data:

We designate the magnitude of the two numbers by A and B. Where the signed numbers are added or subtracted, we find that there are eight different conditions to consider, depending on the sign of the numbers and the operation performed.

Addition: $A + B$; A: Augend; B: Addend
Subtraction: $A - B$; A: Minuend; B: Subtrahend

Operation	Add Magnitude	Subtract Magnitude		
		When $A > B$	When $A < B$	When $A = B$
$(+A) + (+B)$	$+(A + B)$	$+(A - B)$	$-(B - A)$	$+(A - B)$
$(+A) + (-B)$	$+(A - B)$	$-(A - B)$	$+(B - A)$	$+(A - B)$
$(-A) + (+B)$	$-(A + B)$	$+(A - B)$	$-(B - A)$	$+(A - B)$
$(-A) + (-B)$	$-(A - B)$	$-(A - B)$	$+(B - A)$	$+(A - B)$
$(+A) - (+B)$	$+(A - B)$	$+(A - B)$	$-(B - A)$	$+(A - B)$
$(+A) - (-B)$	$+(A + B)$	$-(A - B)$	$+(B - A)$	$+(A - B)$
$(-A) - (+B)$	$-(A + B)$	$-(A - B)$	$+(B - A)$	$+(A - B)$
$(-A) - (-B)$	$-(A - B)$	$-(A - B)$	$+(B - A)$	$+(A - B)$

Table: Addition and Subtraction of Signed-Magnitude Numbers

When the signs of A and B are same, add the two magnitudes and attach the sign of result is that of A. When the signs of A and B are not same, compare the magnitudes and subtract the smaller number from the larger. Choose the sign of the result to be the same as A, if $A > B$ or the complement of the sign of A if $A < B$. If the two magnitudes are equal, subtract B from A and make the sign of the result will be positive.

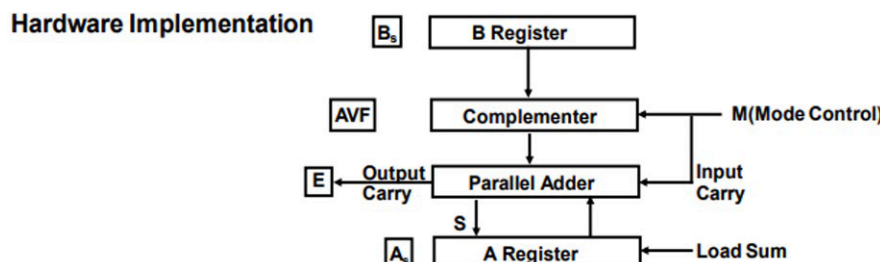
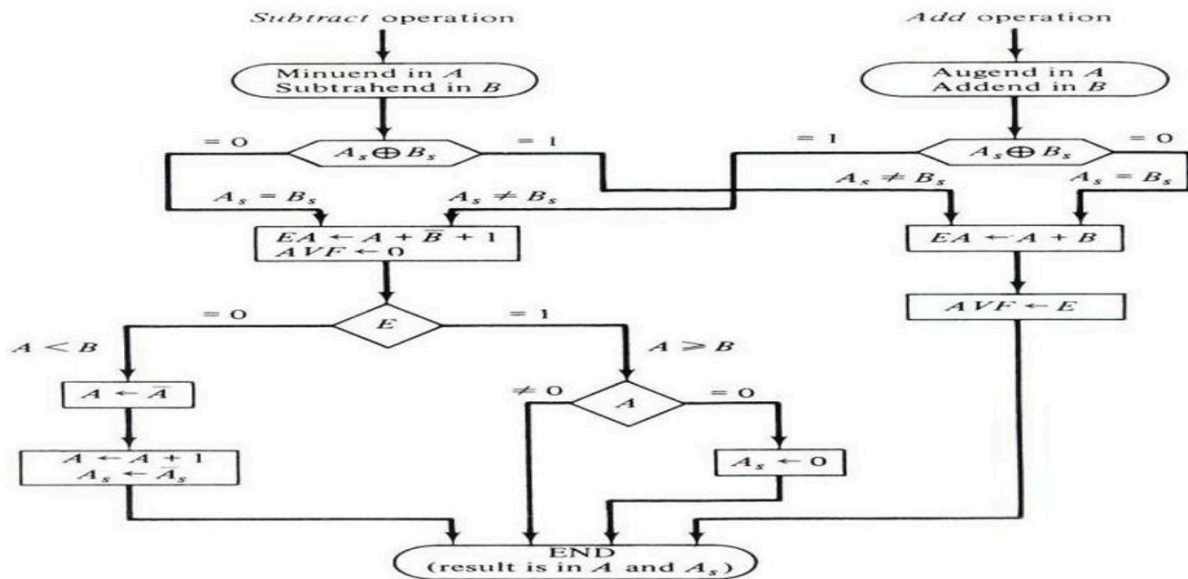


Figure: Hardware Architecture for Addition and Subtraction of Signed-Magnitude Numbers



(b) Subtract 1101 and -1001 using 2's complement subtractions.

13-(-9)

Given: 1101 (minuend) and 1001 (subtrahend)

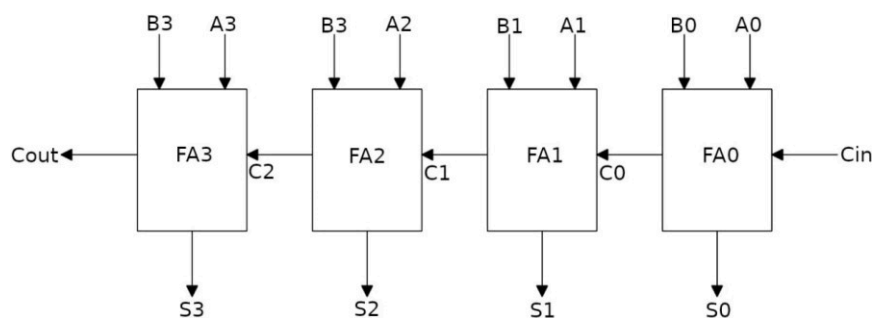
13= 1101 9= 1001

13- (-9) = 13+9

1101 + 1001 = 10110 (22)

3(a) Explain the working of a Ripple carry adder.

A Ripple Carry Adder (RCA) is a simple binary adder used for performing addition of binary numbers. It is made up of multiple Full Adder circuits connected in series, where each Full Adder is responsible for adding two bits and a carry input, then producing a sum and a carry output.



<https://circuitfever.com>

\

Here's how it works step-by-step:

Structure of Ripple Carry Adder

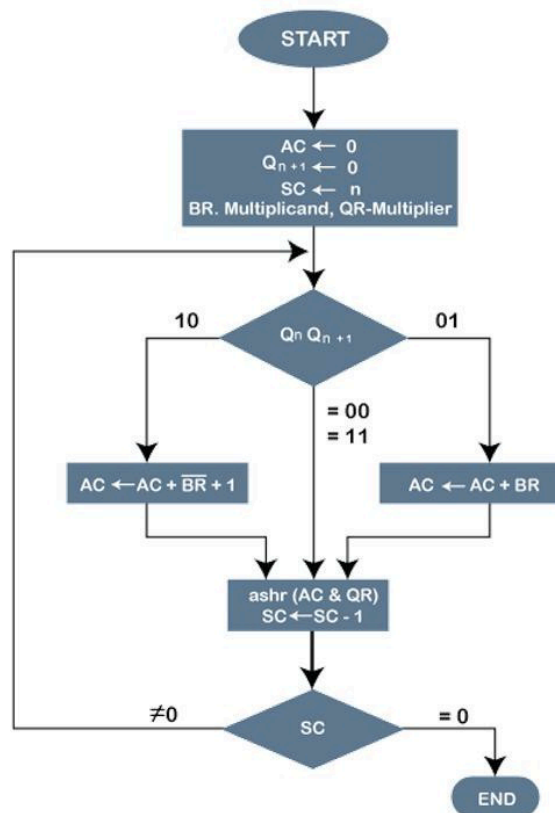
- Full Adder: A Full Adder is a combinational circuit that takes three inputs: two bits

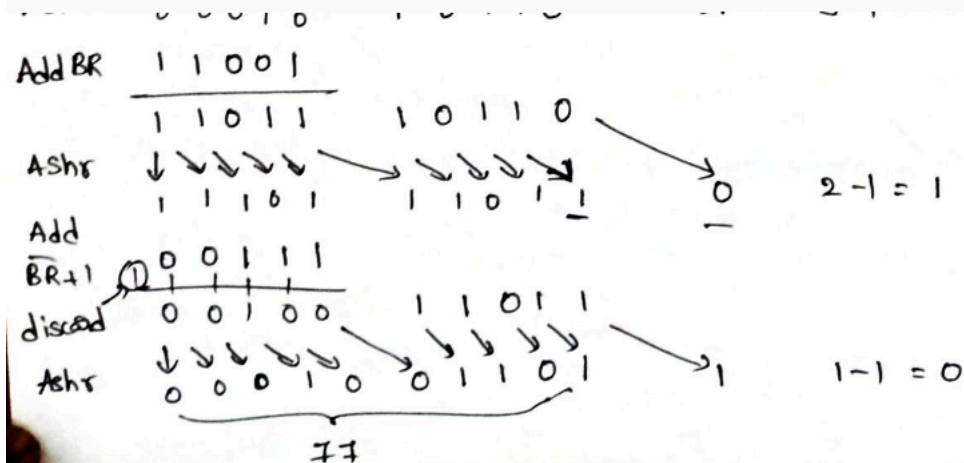
to be added and a carry input from the previous stage. It generates two outputs: a sum and a carry output.

- o Inputs: A, B (the two bits to be added), and Cin (carry input).
- o Outputs: Sum (S) and Cout (carry output).
- o Boolean expressions:
 - Sum (S) = A XOR B XOR Cin
 - Carry-out (Cout) = (A AND B) OR (Cin AND (A XOR B))

(b) Illustrate the steps in Booth multiplication flow chart. Show the step by step signed multiplication of (-7) and (-11) using Booth algorithm

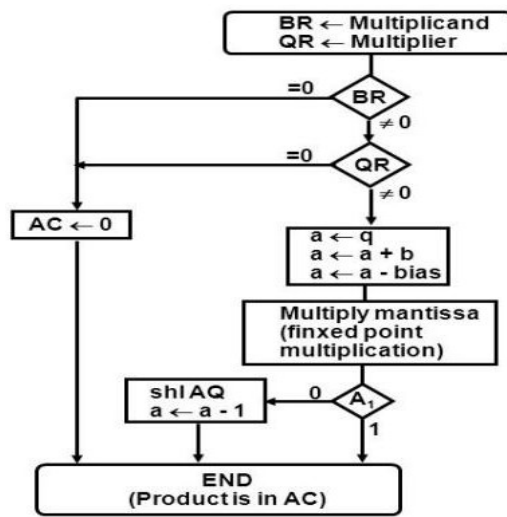
We name the register as A, B and Q, AC, BR and QR respectively. Q_n designates the least significant bit of multiplier in the register QR. An extra flip-flop Q_{n+1} is appended to QR to facilitate a double inspection of the multiplier. The flowchart for the booth algorithm is shown below.





4 Develop flowchart for the Multiplication of floating-point number and illustrate with an example.





Algorithm:

- Check for Zeros
- Add the exponents
- Multiplying the mantissas
- Normalize the

result Step-I: Check for

Zeros

Floating = Mantissa * Base^{exponent}

BR = Multiplicand = 0.12×10^4

QR = Multiplier = 0.23×10^3

BR ≠ 0, QR ≠ 0 we checked for zeros. In BR and QR Register non zeros are there, so we have to check next step.

Step-II: Add the Exponents

$a \leftarrow q$

In BR register the exponent value is considered as

$b = 4$ In QR register the exponent value is

considered as $q = 3$

i.e $a \leftarrow q \Rightarrow a = 3$

$a \leftarrow a + b \Rightarrow a = 3 + 4 = 7$

Step-III: Multiplying the Mantissas

Mantissa in BR register is

0.12 Mantissa in QR

register is 0.23 So,

$0.12 \times 0.23 = 0.0276$



AQ=0.0276

Step-IV: Normalize the Result:

After Multiplying the Mantissas, we have to check $A_1=0$ or $A_1=1$

AQ=0.0276

Here $A_1=0$ we have to perform shift left

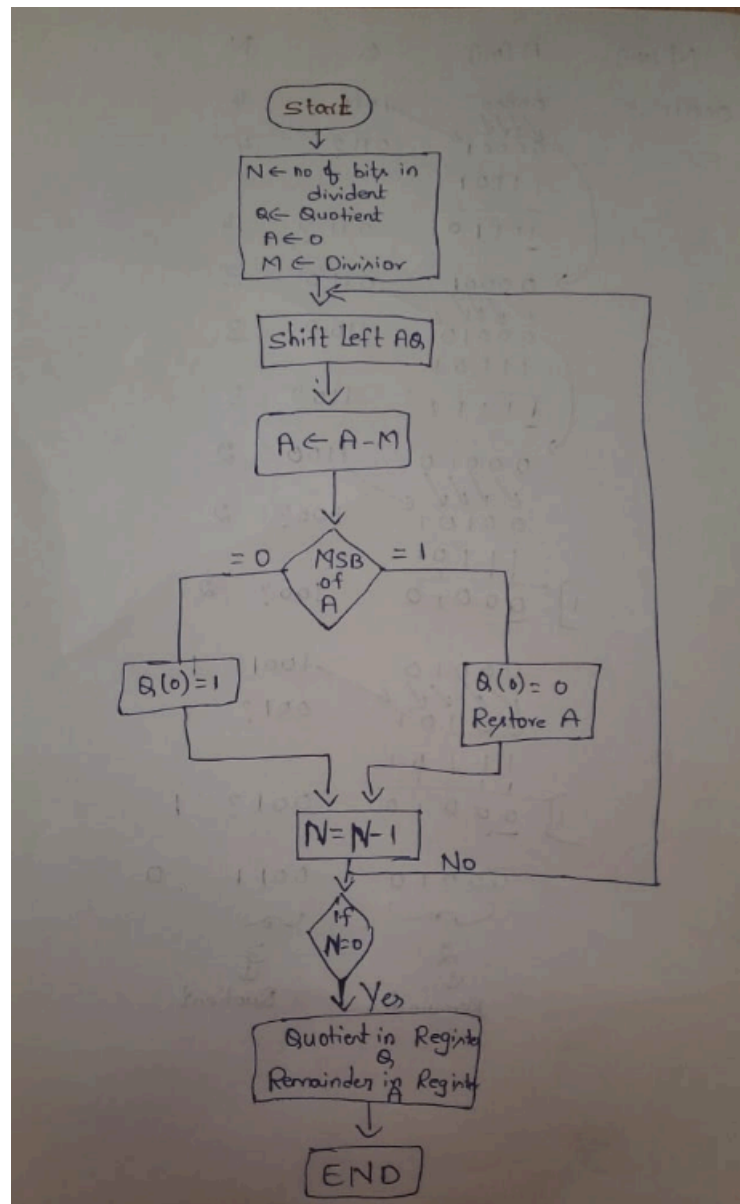
AQ AQ=0.0276

=0.0276 [After shift left]

$a \leftarrow a-1=7-1=6$ [Product=0.276*10⁶]

5 Develop and discuss the Flow chart for Division of numbers Give the step by-step procedure to Divide 11 with 3 find the results

Division Flowchart:



$M(N+1)$	$A(N+1)$	Q	N
00011	00000	1011	4
	00001	011?	4
	11101		
	11110	011?	4
	00001	0110	3
	00010	110?	3
	11101		
	11111	110?	3
	00010	1100	2
	00101	100?	2
	11101		
1	00010	100?	2
	00010	1001	1
	00101	001?	1
	11101		
1	00010	001?	1
	00010	0011	0
	2	3	
	Remainder	Quotient	



6 Develop flow chart for the addition/subtraction of floating-point number and illustrate with an example.

Floating Point Addition/Subtraction:

During Addition or subtraction the two floating point operands are in AC and BR. The sum or difference is formed in the AC.

The algorithm can be into four consecutive parts:

- Check for Zeros
- Align the mantissas
- Add or subtract the mantissas.
- Normalize the result

A floating-point number that is zero cannot be normalized.

If this number is used during the computation, the result may also be zero.

Instead of checking for zeros during the normalization process we check for zeros at the beginning and terminate the process if necessary.

The alignment of the mantissas must be carried out prior to their operation. After the mantissas are added or subtracted the result may be normalized.

Example:

The image shows handwritten notes on a piece of paper, divided into two columns by a vertical line. The left column is titled 'check for zero's (Addition)' and the right column is titled 'check for zero's (subtraction)'. Both columns have sub-headers 'AC' and 'BR'.

Left Column: check for zero's (Addition)

AC BR

Addition:

Case 1: AC=0 BR=0.686
 $AC + BR = 0 + 0.686$
 $= 0.686$

Case 2: AC=0.686 BR=0
 $AC + BR = 0.686$

Step-2: Align the Mantissa

Ex:- 0.583123×10^3
 0.123000×10^{-1}

Case 1: -1 to +3
 $3 - (-1) = 4$
 0.123000×10^{-1}

Right Column: check for zero's (subtraction)

AC BR

Subtraction:

Case 1: AC=0.686 BR=0
 $AC - BR = 0.686 - 0$
 $= 0.686$

Case 2: AC=0 BR=0.686
 $AC - BR = 0 - 0.686 = -0.686$

Step-2:

Ex:- 0.583123×10^3
 0.123000×10^{-1}

Case 1: -1 to 3
 $3 - (-1) = 4$
 0.123000×10^{-1}



7 Explain the fundamental concept in processor organization?

The fundamental concept in processor organization revolves around how the internal components of a central processing unit (CPU) are structured and interact to execute instructions. It focuses on the design and operation of the hardware that carries out the necessary tasks for computing. Key components of processor organization include:

1. Registers

- These are small, fast storage locations within the CPU used to hold data and instructions that are being processed.
- Common registers include the Program Counter (PC), which holds the address of the next instruction to be executed, and Accumulator (AC), which temporarily stores intermediate results during calculations.

2. Control Unit (CU)

- The Control Unit manages the execution of instructions by directing the operation of the processor.
- It generates control signals that coordinate the activities of the various parts of the CPU, such as the ALU, memory, and registers, and ensures the proper flow of data.

3. Arithmetic and Logic Unit (ALU)

- The ALU is responsible for performing mathematical (addition, subtraction, etc.) and logical (AND, OR, NOT, etc.) operations on the data provided by the registers.
- It forms the computational core of the processor.

4. Memory Organization

- Processors interact with memory to fetch instructions and store intermediate results. Memory can be divided into different levels such as cache memory, main memory (RAM), and secondary storage.
- The processor uses the address bus to access specific locations in memory, and the data bus to transfer data.

5. Bus System

- The bus system connects different components of the processor (such as registers, ALU, and memory), allowing data, addresses, and control signals to be transferred between them.
- There are typically three types of buses: data bus (for transferring data), address bus (for specifying memory locations), and control bus (for signaling).

6. Instruction Cycle



- The processor executes instructions in a repetitive cycle, often referred to as the fetch- decode-execute cycle:
 1. Fetch: The processor retrieves the next instruction from memory.
 2. Decode: The control unit decodes the instruction and determines the required action.
 3. Execute: The ALU performs the operation (arithmetic or logical), or the instruction is executed by other components

7. Clock

- The clock provides a timing signal that synchronizes the operations of the processor. The clock speed determines how many instructions the processor can execute in a given period.

8. Execution of Instruction

- The processor executes the instructions sequentially, unless directed otherwise (such as by a jump or branch instruction).
- Each instruction has an opcode (operation code), which specifies the operation to be performed, and operand(s), which specify the data or addresses involved in the operation.

8 Describe the execution of a complete instruction?

The execution of a complete instruction in a computer system follows a sequence of steps known as the fetch-decode-execute cycle or instruction cycle. This process involves several stages where data is retrieved, processed, and stored, allowing the computer to carry out tasks. Here's a breakdown of the stages:

1. Fetch:

- Fetch the instruction from memory: The Program Counter (PC) holds the address of the next instruction to be executed. The instruction is fetched from the memory location indicated by the PC.
- Increment the Program Counter: After fetching the instruction, the Program Counter is incremented to point to the next instruction, ensuring that instructions are executed sequentially (unless a jump or branch is specified).

2. Decode:

- Instruction Decoding: The fetched instruction is sent to the Instruction Decoder. The decoder interprets the opcode (operation code), which specifies the operation to be performed (e.g., addition, subtraction, load, store).
- Operand Identification: If the instruction requires operands (such as data to

operate on), the decoder identifies where the operands are located (in registers, memory, etc.). These operands might also be fetched at this stage if not already available.

3. Execute:

- **Perform the Operation:** The Arithmetic Logic Unit (ALU) or other parts of the processor perform the operation specified by the decoded instruction. This could involve mathematical computations, logical operations, data movement, etc.
- **Update Flags:** If necessary, the processor's status flags (such as zero, carry, overflow, etc.) are updated to reflect the result of the operation (e.g., if a result is zero or if a carry occurred).

4. Memory Access (if needed):

- **Read/Write to Memory:** If the instruction involves reading from or writing to memory (for example, in load or store operations), the data is either fetched from or written to the memory during this stage.

5. Write Back:

- **Store the Result:** The result of the execution (e.g., an arithmetic operation or data retrieval) is written back to a register or memory, depending on the instruction.
- **Update the Program Counter:** In some cases, if the instruction involves branching (like jumps or conditional branches), the Program Counter is updated to reflect the new address to jump to, instead of continuing sequentially.

6. Repeat: The process repeats for each instruction in the program, starting with the fetch phase for the next instruction.

Example of Instruction Execution:

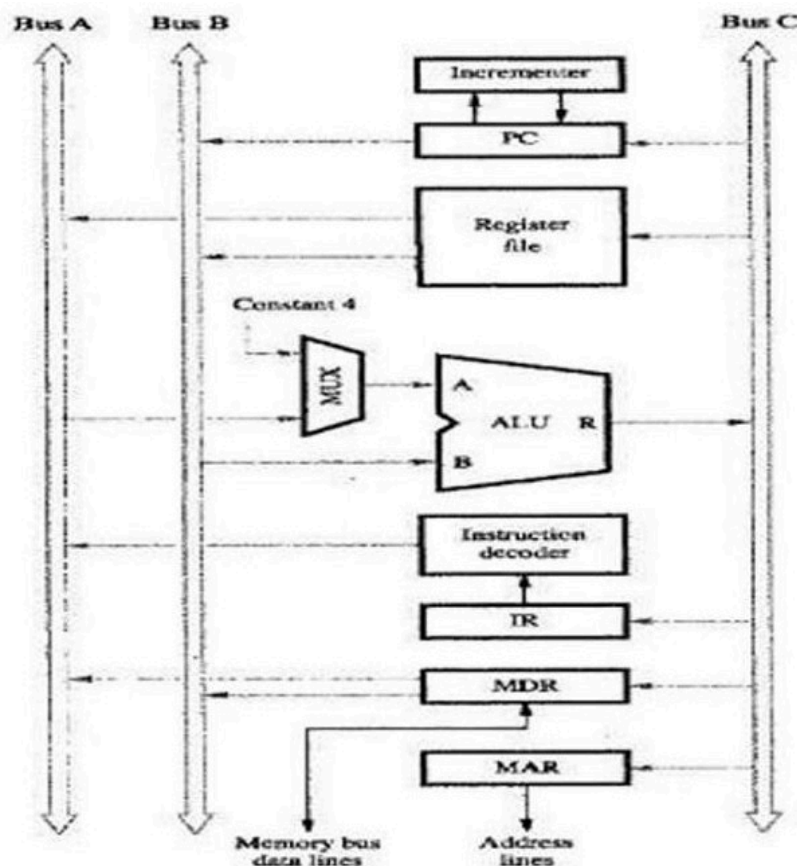
Suppose the instruction is ADD R1, R2, R3 (add the contents of registers R2 and R3, and store the result in R1).

1. **Fetch:** The instruction ADD R1, R2, R3 is fetched from memory using the address in the Program Counter.
2. **Decode:** The opcode ADD is decoded to indicate an addition operation. The operands are identified as the contents of registers R2 and R3.
3. **Execute:** The ALU adds the contents of R2 and R3.
4. **Memory Access:** No memory access is required in this case, as the operation is register- based.
5. **Write Back:** The result of the addition is written back to register R1.
6. **Repeat:** The Program Counter is updated, and the next instruction is fetched.

9(a) Explain the multiple bus organization.

Multiple Bus Organization:

- All general-purpose registers are combined into a single block called the register file.
- Register-file has 3 ports. There are 2 outputs allowing the contents of 2 different registers to be simultaneously placed on the buses A and B.
- Register-file has 3 ports.
 - 1) Two output-ports allow the contents of 2 different registers to be simultaneously placed on buses A & B.
 - 2) Third input-port allows data on bus C to be loaded into a third register during the same clock-cycle.
- Buses A and B are used to transfer source-operands to A & B inputs of ALU.
- Result is transferred to destination over bus C.
- Incrementer-unit is used to increment PC by 4.



(b) Differentiate between Hardwired Control and Micro-programmed control.

S.No	Hardwired Control	Micro-programmed Control
1	Hardwired control unit generates the control signals needed for the processor using logic circuits	Microprogrammed control unit generates the control signals with the help of micro instructions stored in control memory
2	Hardwired control unit is faster when compared to microprogrammed control unit as the required control signals are generated with the help of hardwares	This is slower than the other as micro instructions are used for generating signals here
3	Difficult to modify as the control signals that need to be generated are hard wired	Easy to modify as the modification need to be done only at the instruction level
4	More costlier as everything has to be realized in terms of logic gates	Less costlier than hardwired control as only micro instructions are used for generating control signals
5	It cannot handle complex instructions as the circuit design for it becomes complex	It can handle complex instructions
6	Only limited number of instructions are used due to the hardware implementation	Control signals for many instructions can be generated
7	Used in computer that makes use of Reduced Instruction Set Computers (RISC)	Used in computer that makes use of Complex Instruction Set Computers (CISC)

10 What is Hardwired Control? Explain in detail with a neat diagram.

Hardwired Control: To execute instructions, the processor must have some means of generating the control signals needed in the proper sequence. Computer designers use a wide variety of techniques to solve this problem. The approaches used fall into one of two categories: hardwired control and micro programmed control.

The required control signals are determined by the following information:

- Contents of the control step counter
- Contents of the instruction register
- Contents of the condition code flags
- External input signals, such as MFC and interrupt requests



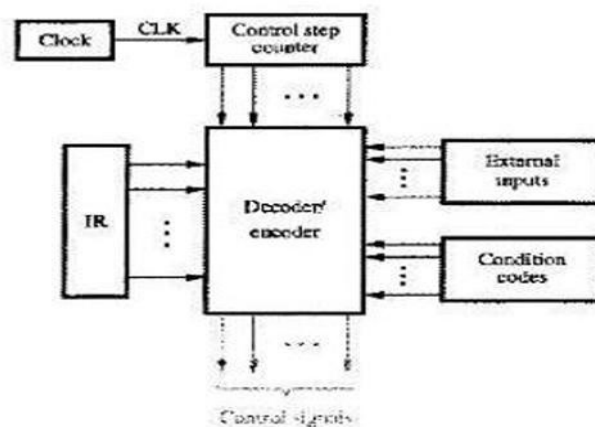


Fig: Control unit organization

To gain insight into the structure of the control unit, we start with a simplified view of the hardware involved. The decoder/encoder block in Figure is a combinational circuit that generates the required control outputs, depending on the state of all its inputs. By separating the decoding and encoding functions, we obtain the more detailed block diagram in Figure. The step decoder provides a separate signal line for each step, or time slot, in the control sequence. Similarly, the output of the instruction decoder consists of a separate line for each machine instruction.

11 What is micro programmed Control? Explain in detail with a neat diagram.

Multi programmed Control: Microprogrammed control signals are generated by a program similar to machine language programs. ALU is the heart of any computing system, while Control unit is its brain. The design of a control unit is not unique; it varies from designer to designer. Some of the commonly used control logic design methods are:

- Sequence Reg & Decoder method
- Hard-wired control method
- PLA control method
- Micro-program control method

To support microprogram branching, the organization of the control unit should be modified as shown in Figure. The starting address generator block of Figure becomes the starting and branch address generator. This block loads a new address into the μ PC when a microinstruction instructs it to do so. To allow implementation of a conditional branch, inputs to this block consist of the external inputs and condition codes as well as the contents of the instruction register. In this control unit, the μ PC is incremented every time a new microinstruction is fetched from the micro program memory, except in the following situations:

1. When a new instruction is loaded into the IR, the μ PC is loaded with the starting address of the micro routine for that instruction.



2. When a Branch microinstruction is encountered and the branch condition is satisfied, the μ PC is loaded with the branch address. When an End microinstruction is encountered, the μ PC is loaded with the address of the first CW in the micro routine for the instruction.

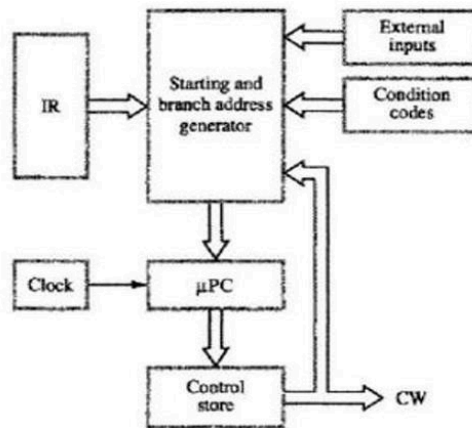


Fig: Organization of the control unit to allow conditional branching in the microprogram





SIDDHARTH INSTITUTE OF ENGINEERING & TECHNOLOGY

(AUTONOMOUS)

(Approved by AICTE, New Delhi & Affiliated to JNTUA, Ananthapuramu)

(Accredited by NBA for Civil, EEE, Mech., ECE & CSE)

(Accredited by NAAC with 'A+' Grade)

Puttur -517583, Tirupati District, A.P. (India)

UNIT- IV PART

-A (2 MARKS)

1(a) What is the need of memory?

Memory refers to the electronic components used by computers to store data and instructions that can be accessed quickly by the processor. It allows you to store and retrieve information for various computing tasks.

(b) Define main memory and auxiliary memory

Main Memory and Auxiliary Memory are two fundamental types of memory used in computer systems for storing data, programs, and instructions. They differ in their purpose, characteristics, and usage.

Main Memory (Primary Memory): Main memory is the high-speed memory directly accessible by the CPU. It stores data and instructions currently in use for quick access.

☐ Examples:

- Random Access Memory (RAM)
- Cache memory

Auxiliary Memory (Secondary Memory): Auxiliary memory is a non-volatile memory used to store data and programs that are not currently in use by the CPU.

☐ Examples:

- Hard Disk Drives (HDDs)
- Solid-State Drives (SSDs)
- Optical Discs (CDs, DVDs)
- USB Drives
- Magnetic Tapes

(c) Classify main memory and secondary memory?

Main Memory (Primary Memory): Main memory is the part of a computer that directly interacts with the CPU for processing data. It is volatile, meaning data is lost when power is turned off.



Edit with WPS Office

Types of Main Memory:

RAM (Random Access Memory):

- **Dynamic RAM (DRAM):** Slower and needs periodic refreshing.
- **Static RAM (SRAM):** Faster and does not require refreshing.

ROM (Read-Only Memory):

- **PROM (Programmable ROM):** Can be written once.
- **EPROM (Erasable Programmable ROM):** Can be erased using UV light and reprogrammed.
- **EEPROM (Electrically Erasable Programmable ROM):** Can be erased and programmed electrically.

Secondary Memory: Secondary memory is used to store data and programs permanently. It is non-volatile, meaning data is retained even when the computer is turned off.

Types of Secondary Memory:

Magnetic Storage:

- **Hard Disk Drives (HDDs):** Stores large amounts of data using magnetic platters.
- **Floppy Disks (obsolete):** Portable but low-capacity storage.

(d) What is cache memory?

Cache Memory: Cache memory is a high-speed storage layer that provides faster access to frequently used data than main memory (RAM). It is located closer to the CPU, either within the processor itself or on a separate chip, and is used to temporarily store copies of data and instructions that the CPU is likely to use next.

Key Characteristics of Cache Memory:

Speed: It is much faster than RAM, helping to reduce the time needed to access data.

Size: Cache memory is smaller in size compared to RAM and storage, typically ranging from a few kilobytes (KB) to a few megabytes (MB).

(e) Define virtual memory?

Virtual Memory: Virtual memory is a memory management technique used by operating systems to provide an application with the illusion of having a large, contiguous block of memory, even if the physical memory (RAM) is limited. It achieves this by combining the computer's RAM with a portion of the storage drive (e.g., a hard disk or SSD) to create a larger, virtual memory space.



PART-B (10 MARKS)

2. Explain different types of ROM memories in detail?

Read Only Memory: Read Only Memory, is non-volatile and is more like a permanent storage for information. It also stores the bootstrap loader program, to load and start the operating system when computer is turned on. PROM (Programmable ROM), EPROM (Erasable PROM), EEPROM (Electrically Erasable PROM) and Flash Memory are some commonly used ROMs.

1. **Programmable ROM (PROM):** PROM stands for Programmable ROM. It allows the data to be loaded by the user. PROM's provide a faster and the less expensive approach because they can be programmed directly by the user.

2. **Erasable Programmable ROM (EPROM):** EPROM stands for Erasable programmable ROM. It allows the stored data to be erased and new data to be loaded. They can be used in place of ROMs while software is being developed. In this way, memory changes and updates can be easily made.

3. **Electrically Erasable Programmable ROM (EEPROM):** EEPROM stands for Electrically Erasable programmable ROM. A significant disadvantage of EPROMs is that a chip must be physically removed from the circuit for reprogramming and that its entire contents are erased by the ultraviolet light. On the other hand, EEPROMs do not have to be removed to erase the data. Moreover, it is possible to erase the cell contents selectively.

4. **Flash Memory:** Flash memory is secondary memory and so it is not volatile which means it persists the data even if there is not an electrical supply provided. This flash memory works on the principle of EEPROM. EEPROM stands for Electrical Erasable Programmable Read-Only Memory. ROM operation can only one time write and many times read and we can't erase it. But Flash Memory can be erased multiple times and update the data or program integrated into it. So, it gives flexibility to the updation of the program but ROM has no such type of feature.

3. Categorize the semiconductor RAM in detail.

Semiconductor RAM Memory: Semiconductor RAM (Random Access Memory) is a type of computer memory that can be read from and written to many times. It is used to store data that is being processed by the computer, such as program instructions and temporary data. RAM is volatile, which means that it loses its contents when the power is turned off.



Types of Semiconductor RAM

There are two main types of semiconductor RAM:

Static RAM (SRAM): SRAM cells use a flip-flop circuit to store data. Flip-flops are made up of two transistors that are connected in a feedback loop. When one transistor is turned on, the other is turned off, and vice versa. This creates two stable states, which can be used to represent the binary digits 0 and 1. SRAM cells are very fast and reliable, but they are also relatively expensive to manufacture.

Dynamic RAM (DRAM): DRAM cells use a capacitor to store data. Capacitors are made up of two metal plates that are separated by a dielectric material. When a voltage is applied to the plates, the capacitor stores an electrical charge. The amount of charge stored depends on the voltage, and this can be used to represent the binary digits 0 and 1. DRAM cells are less expensive to manufacture than SRAM cells, but they are also slower and less reliable.

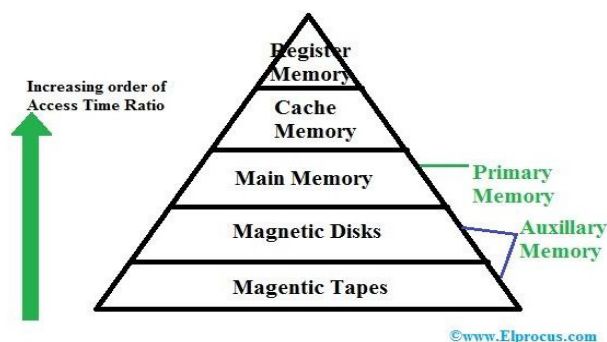
Applications of Semiconductor RAM

Semiconductor RAM is used in a wide variety of electronic devices, including:

- Personal computers
- Servers
- Workstations
- Laptops
- Tablets
- Smartphones
- Digital cameras

4(a) Describe about memory hierarchy concept in detail?

Memory hierarchy organizes memory components based on their access and response times. Moreover, several levels of memory exist, each with different access speeds and performance. This structure ensures that frequently accessed information is obtained faster than less frequently used data.



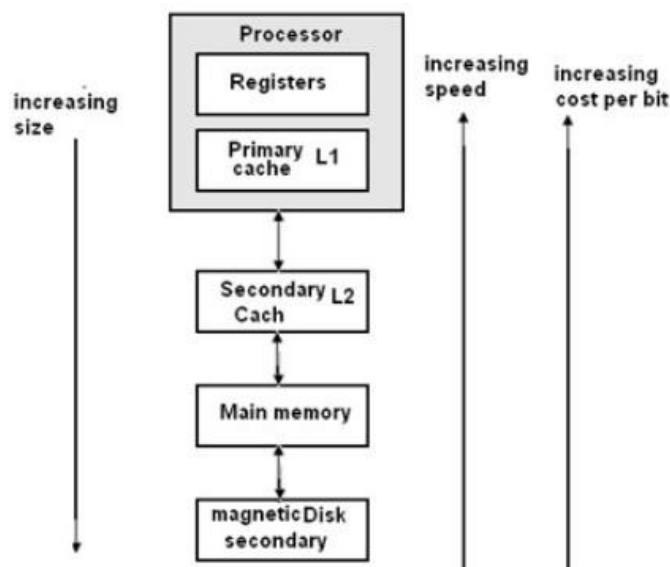
Register Memory: The total memory capacity of a computer can be visualized by the hierarchy of components. The memory hierarchy consists of all storage devices containing in a computer system from the slow auxiliary memory to fast main memory and to smaller cache memory. The auxiliary memory access time is generally thousand times that of the main memory and hence it is a bottom of the hierarchy.

Cache Memory: It is used to store program data which is currently being executed in the CPU.

Main Memory: The memory unit communicates directly with in the CPU, Auxiliary memory and cache memory is also called main memory. It is the central storage unit of the computer system. It is a large and fast memory used to store data during the computer operations. Main memory is made up of RAM and ROM.

Auxiliary Memory: The devices that provide backup storage are called Auxiliary memory. Magnetic disks and magnetic tapes are commonly used for Auxiliary devices. The other devices are used as Auxiliary memory are magnetic drums, magnetic bubble memory and optical disk. It is not directly accessible to the CPU and it is accessed using the Input/Output channels.

4(b) Discuss the speed, size and cost of a memory?

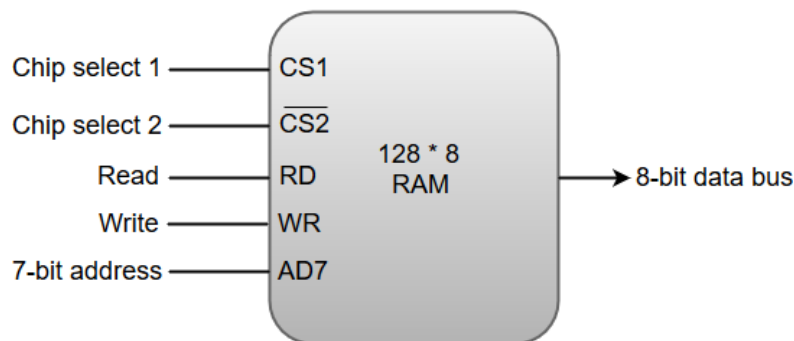


- An ideal memory would be fast, large and Inexpensive.
- It is very fast memory can be implemented if SRAM chips are used (very expensive, each cell has six transistors)
- The Dynamic RAM chips have much simpler basic cells and thus are much less expensive.
- Some memories are very slower.
- The faster access is to data held in processor registers.
- Therefore, if we consider the registers to be part of the memory hierarchy then the processor registers are at the top in terms of the speed of access.
- The registers provide only a small portion of the required memory.
- The processor cache holds copies of instructions and data stored in a much larger memory that is provided externally.
- Two levels of caches:
 - A primary cache located in the processor chip it referred to as level L1 cache
 - A large secondary cache is placed between the primary cache and the rest of memory referred as level L2.
- In some system both L1 and L2 caches on processor chip.

5(a) Explain 128*8 RAM with block diagram and function table.

A **128 × 8 RAM** refers to a Random Access Memory (RAM) chip with 128 memory locations, each capable of storing 8 bits (1 byte) of data. Here, "128 × 8" indicates the organization of the memory, where there are 128 addressable rows (memory locations) and 8 bits (1 byte) per row.

Typical RAM chip:



- A 128 * 8 RAM chip has a memory capacity of 128 words of eight bits (one byte) per word. This requires a 7-bit address and an 8-bit bidirectional data bus.
- The 8-bit bidirectional data bus allows the transfer of data either from memory to CPU during a read operation or from CPU to memory during a write operation.
- The read and write inputs specify the memory operation, and the two chip select (CS) control inputs are for enabling the chip only when the microprocessor selects it.
- The bidirectional data bus is constructed using three-state buffers.
- The output generated by three-state buffers can be placed in one of the three possible states which include a signal equivalent to logic 1, a signal equal to logic 0, or a high-impedance state.



The following function table specifies the operations of a 128 * 8 RAM chip.

CS1	$\overline{\text{CS2}}$	RD	WR	Memory function	State of data bus
0	0	x	x	Inhibit	High-impedance
0	1	x	x	Inhibit	High-impedance
1	0	0	0	Inhibit	High-impedance
1	0	0	1	Write	Input data to RAM
1	0	1	x	Read	Output data to RAM
1	1	x	x	Inhibit	High-impedance

From the functional table, we can conclude that the unit is in operation only when CS1 = 1 and CS2 = 0. The bar on top of the second select variable indicates that this input is enabled when it is equal to 0.

5(b) Distinguish between SRAM & DRAM?

SRAM	DRAM
SRAM stands for Static Random Access Memory.	DRAM stands for Dynamic Random Access Memory.
SRAM stores information with the help of transistors.	DRAM stores data using capacitors.
In SRAM, capacitors are not used which means refresh is not needed.	In DRAM, contents of a capacitor need to be refreshed periodically.
SRAM provides faster speed of data read/write.	DRAM provides slower speed of data read/write.
SRAM consumes more power.	DRAM consumes less power.
SRAM has long data life.	DRAM has short data life.
SRAM is expensive.	DRAM is less expensive.
SRAM is a low-density device.	DRAM is a high-density device.
SRAMs are used as cache memory in computer and other computing devices.	DRAMs are used as main memory in computer systems.



6. Explain how memories connected with CPU with diagram.

Memory Connection to CPU: RAM and ROM chips are connected to a CPU through the data and address buses. The low-order lines in the address bus select the byte within the chips and other lines in the address bus select a particular chip through its chip select inputs. The connection of memory chips to the CPU is shown in Figure.

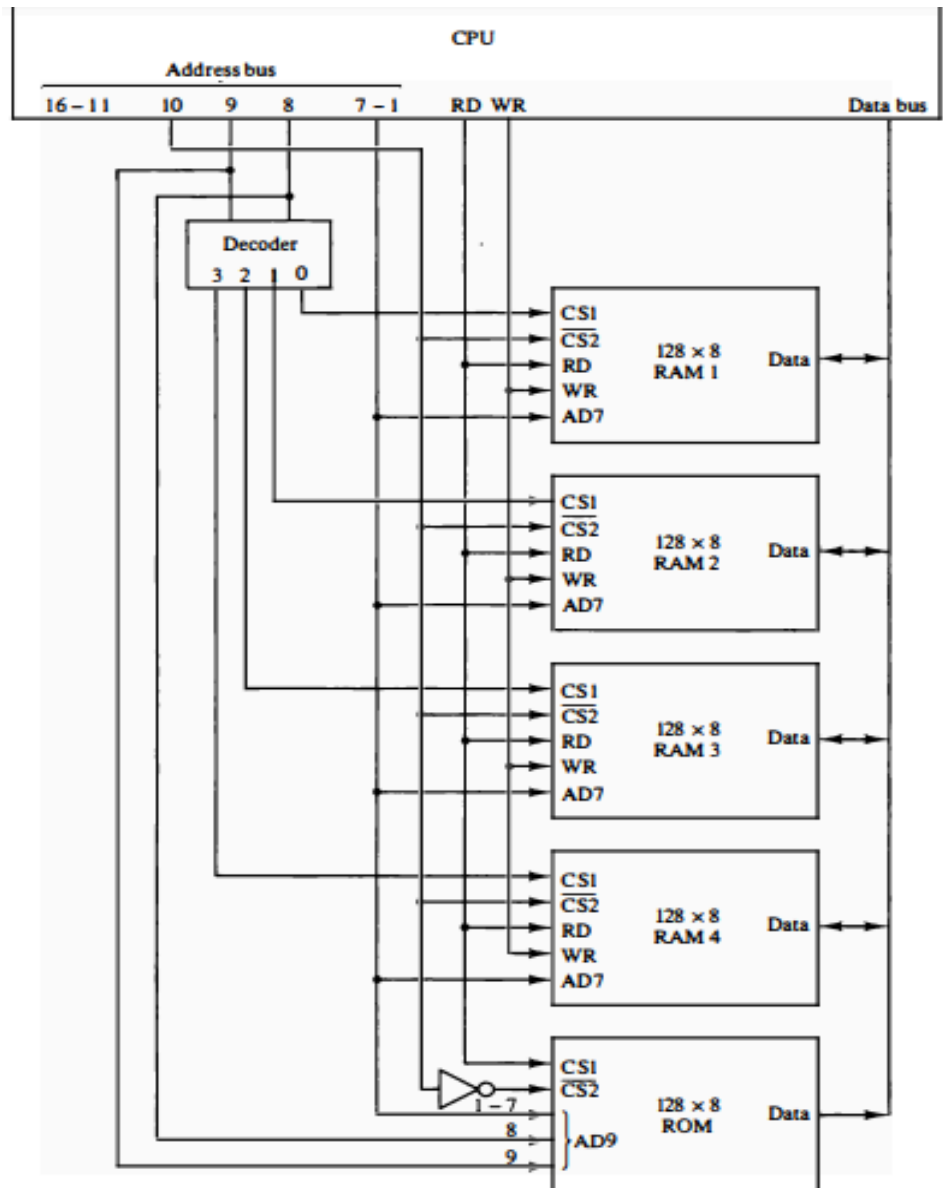


Figure 4 Memory connection to the CPU.

This configuration gives a memory capacity of 512 bytes of RAM and 512 bytes of ROM. It implements the memory map as shown in below Table.

TABLE 1 Memory Address Map for Microcomputer

Component	Hexadecimal address	Address bus									
		10	9	8	7	6	5	4	3	2	1
RAM 1	0000-007F	0	0	0	x	x	x	x	x	x	x
RAM 2	0080-00FF	0	0	1	x	x	x	x	x	x	x
RAM 3	0100-017F	0	1	0	x	x	x	x	x	x	x
RAM 4	0180-01FF	0	1	1	x	x	x	x	x	x	x
ROM	0200-03FF	1	x	x	x	x	x	x	x	x	x

Each RAM receives the seven low-order bits of the address bus to select one of 128 possible bytes. The particular RAM chip selected is determined from lines 8 and 9 in the address bus. This is done through a 2 X 4 decoder whose outputs go to the CS1 inputs in each RAM chip. Thus, when address lines 8 and 9 are equal to 00, the first RAM chip is selected. When 01, the second RAM chip is selected, and so on. The RD and WR outputs from the microprocessor are applied to the inputs of each RAM chip. The selection between RAM and ROM is achieved through bus line 10. The RAMs are selected when the bit in this line is 0, and the ROM when the bit is 1. Address bus lines 1 to 9 are applied to the input address of ROM without going through the decoder. The data bus of the ROM has only an output capability, whereas the data bus connected to the RAMs can transfer information in both directions.

7(a) What is cache memory? What is hit and miss in the cache memory.

Cache Memory is a small, high-speed storage located between the CPU (Central Processing Unit) and the main memory (RAM). It stores frequently accessed data and instructions to improve the speed of data retrieval and processing. Cache memory is faster than the main memory and is designed to reduce the time it takes for the CPU to access data from RAM.

There are typically three levels of cache memory:

1. **L1 Cache:** Directly integrated into the CPU and very fast, but has a small capacity.
2. **L2 Cache:** Larger than L1 but slightly slower; often shared between multiple CPU cores.
3. **L3 Cache:** Even larger and slower than L2, often shared by all cores in multi-core processors.

Cache Hit and Cache Miss



- **Cache Hit:** A cache **hit** occurs when the CPU finds the requested data or instruction in the cache memory. This results in faster access because the CPU can immediately use the data stored in the cache without needing to fetch it from the slower main memory (RAM). The hit rate is the percentage of accesses that result in a hit.
- **Cache Miss:** A cache **miss** happens when the requested data is not found in the cache. In this case, the CPU must retrieve the data from the slower main memory, which results in a longer delay in processing. There are several types of cache misses:
 - **Compulsory Miss:** Occurs when data is accessed for the first time and is not yet in the cache.
 - **Capacity Miss:** Happens when the cache is full, and older data needs to be replaced to make room for new data.
 - **Conflict Miss:** Occurs when multiple pieces of data that need to be cached are mapped to the same location in the cache, even if there is space elsewhere in the cache.

7(b) List and explain different mapping in Cache memory

The basic characteristic of cache memory is its fast access time. Therefore, very little or no time must be wasted when searching for words in the cache. The transformation of data from main memory to cache memory is referred to as a mapping process.

Three types of mapping procedures are of practical interest when considering the organization of cache memory:

1. Associative mapping
 2. Direct mapping
 3. Set-associative mapping
1. **Associative Mapping:**



1. Associative mapping

* The size of memory is $32\text{KB} \times 12$.

$$2^5 \times 2^{10} \rightarrow 12 \text{ bits (word length)}$$

$$2^{15} \rightarrow 12 \text{ bits}$$

* The fastest and more flexible cache organization uses an Associative memory. Its organization shown in below.

CPU Address (15 bits)

↓
Argument
Register

← address →	← Data →
01000	3450
02777	6710
22345	1234

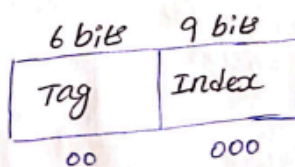


- * The associative memory stores both the address and data of the memory word. This permits any location in cache to store any word from the main memory.
- * This diagram shows three words presently stored in the cache. The address value of 15-bits is shown as 5-digit octal number and its corresponding 12-bit word is shown as a 4-digit octal number.
- * A CPU address of 15-bits is placed in the argument register and the associative memory is searched for a matching address.
- * If the address is found, the corresponding 12-bit data is read and sent to the CPU.
- * If no match occurs, the main memory is accessed for the word. The address-data pair is then transferred to the associative cache memory.
- * Assume cache memory is full (512x12 is full) then it will replace one of the word from main memory.
- * The selection word based is depends on replacement algorithm like FIFO, LRU algorithms etc.

2. Direct Mapping:

2. Direct mapping

- * CPU generated address is divided into 2 parts. They are Tag & Index. Its representation is

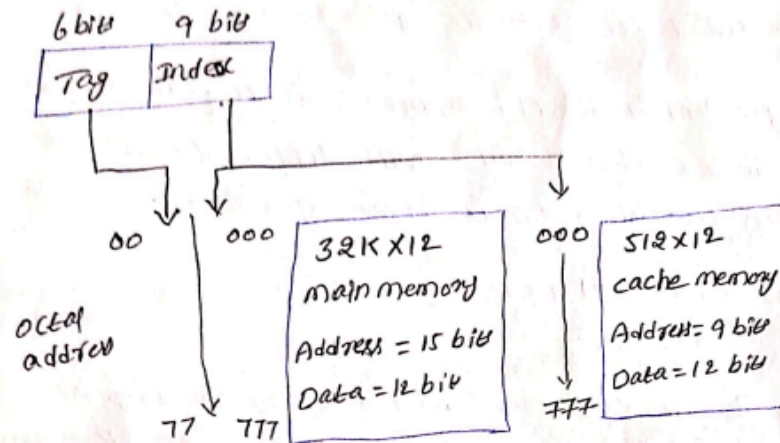


000 - 3 octal digits
 3 3 3 3x3
 digit

- * This CPU generated address consists of 15 bits.
- * The Index field (9 bits) consists of 9 bits and Tag field consists of 6 bits. Total bits are 15 bits.
- * Each address consists of 3 octal numbers. So here Index field has 3 octal digits (000) each and every digit specifies the 3 octal bits. So, Index field has $3 \times 3 = 9$ bits.



- * Tag field consists of 2 octal digits. Each and every octal digit specify the 3 octal digits. so $3 \times 2 = 6$ bits
- * Addressing relationship between main memory and cache memory as shown in below figure.



- * Cache memory contains the 3-bit octal digits and main memory contains the 5-bit octal digits.

main memory		cache memory	
Address	Data	Index address	Tag + Data
00000	1220	00	1220
00777	2340		
01000	3450		
01777	4560		
02000	5670		
02777	6710	02	6710

- * From the above the main memory consists of 5 address bits. These are transferred to the cache memory. Now the cache memory allocated the 5-bit first two bits are Tag and remaining 3-bits are Index address and the data of main memory is loaded into cache memory data.



3. Set Associative Mapping:

3. Set Associative Mapping

- * In direct mapping technique, the main drawback is the performance or hit ratio is drastically reduced.
- * This problem is overcome with the help of set associative mapping.
- * The major advantage of this mapping is in a single index, we can store multiple words.

Cache memory

Index	Tag	Data	Tag	Data
000	00	1220	01	1234
777	02	6712	04	4567

- * In each single index, we can store multiple tags (or) words.
- * For example, the index values are same when we compare the main-memory and cache memory. So here same index address, then the cache memory allocated data is used by the CPU and this CPU executes the word.

8(a) Compare the various cache mapping techniques.

The basic characteristic of cache memory is its fast access time. Therefore, very little or no time must be wasted when searching for words in the cache. The transformation of data from main memory to cache memory is referred to as a mapping process. Three types of mapping procedures are of practical interest when considering the organization of cache memory:

1. Associative mapping
2. Direct mapping
3. Set-associative mapping



1. Associative Mapping:

1. Associative mapping:-

- * The size of memory is $32 \text{ KB} \times 12$.
 $2^5 \times 2^{10} \rightarrow 12 \text{ bits (word length)}$
 $2^{15} \rightarrow 12 \text{ bits}$
- * The fastest and more flexible cache organization uses an Associative memory. Its organization shown in below.

CPU Address (15 bits)

Argument Register

address	Data
01000	3450
02111	6710
22345	1234

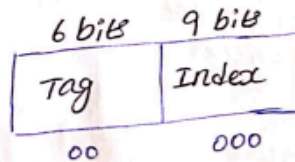
- * The associative memory stores both the address and data of the memory word. This permits any location in cache to store any word from the main memory.
- * This diagram shows three words presently stored in the cache. The address value of 15-bits is shown as 5-digit octal number and its corresponding 12-bit word is shown as a 4-digit octal number.
- * A CPU address of 15-bits is placed in the argument register and the associative memory is searched for a matching address.
- * If the address is found, the corresponding 12-bit data is read and sent to the CPU.
- * If no match occurs, the main memory is accessed for the word. The address-data pair is then transferred to the associative cache memory.
- * Assume cache memory is full (512×12 is full) then it will replace one of the word from main memory.
- * The selection word based is depends on replacement algorithm like FIFO, LRU algorithm, etc.



2. Direct Mapping:

2. Direct mapping

- * CPU generated address is divided into 2 parts. They are Tag & Index. Its representation is

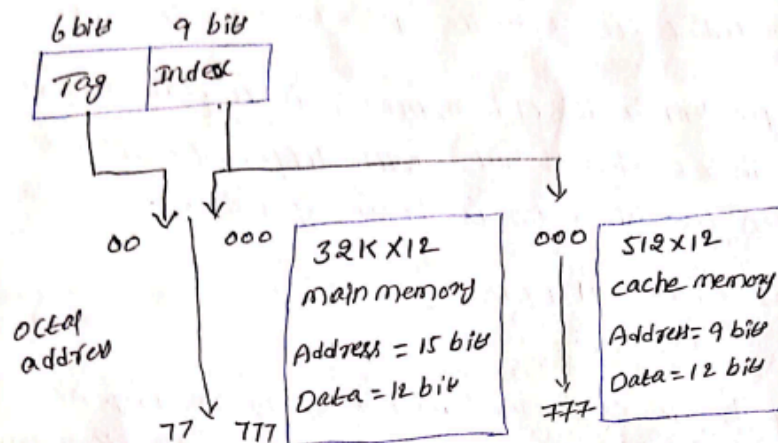


000 - 3 octal digits
3 1 1 3x3
digit

- * This CPU generated address consists of 15 bits.
- * The Index field (9 bits) consists of 9 bits and Tag field consists of 6 bits. Total bits are 15 bits.
- * Each address consists of 3 octal numbers. So here Index field has 3 octal digits (000) each and every digit specifies the 3 octal bits. So Index field has $3 \times 3 = 9$ bits.

- * Tag field consists of 2 octal digits. Each and every octal digit specifies the 3 octal digits. So $3 \times 2 = 6$ bits

- * Addressing relationship between main memory and cache memory is shown in below figure.



- * Cache memory contains the 3-bit octal digits and main memory contains the 5-bit octal digits.

main memory		cache memory	
Address	Data	Index address	Tag Data
00000	1220	000	00 1220
00777	2340		
01000	3450		
01777	4560		
02000	5670		
02777	6710	777	02 6710

- * From the above the main memory consists of 5 address bits. These are transferred to the cache memory. Now the cache memory allocated the 5-bit first two bits are "Tag" and remaining 3-bits are Index address and the data of main memory is loaded into cache memory Data.

3. Set Associative Mapping:

3. Set Associative mapping

- * In direct mapping technique, the main drawback is the performance or hit ratio is drastically reduced.
- * This problem is overcome with the help of Set Associative mapping.
- * The major advantage of this mapping is in a single index, we can store multiple words.

Index		Tag		Data	
000		00	1220	01	1234
777		02	6712	04	4567

- * In each single index, we can store multiple Tags (or) words.
- * For example, the index values are same when we compare the main-memory and cache memory. So here same index address, then the cache memory allocated data is used by the CPU and this CPU executes the word.



8(b) What are the performance considerations in cache memory?

Performance Considerations:

- Two Key factors in the commercial success of a computer are performance and cost; the best possible performance at the lowest cost is the objective.
- A common measure of success is the price/performance ratio.
- Performance depends on how fast machine instructions can be brought into the processor for execution and how fast they can be executed.

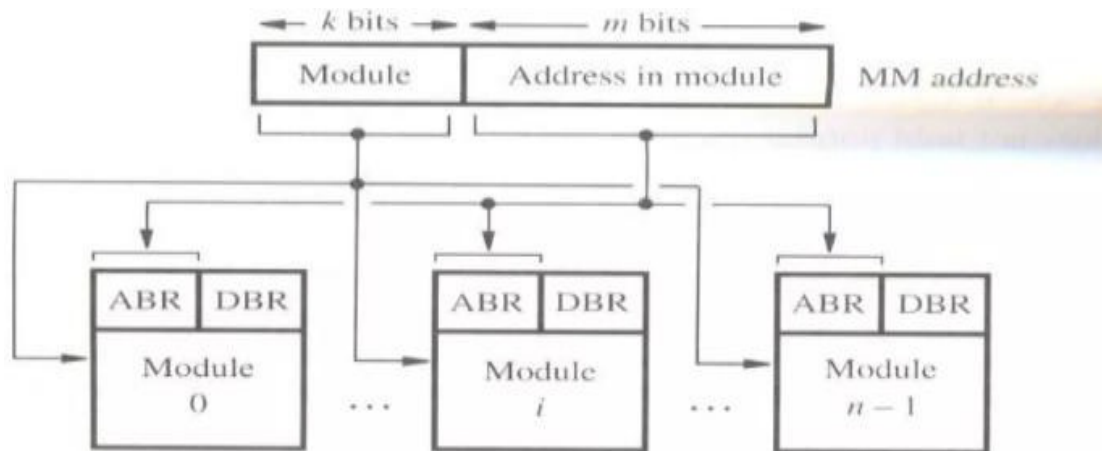
1. Interleaving:

Divides the memory system into a number of memory modules. Each module has its own Address Buffer Register (ABR) and Data Buffer Register (DBR).

Arranges addressing so that successive words in the addresses, the access will be to different modules.

When requests for memory access involve consecutive addresses, the access will be to different modules.

Since parallel access to these modules is possible, the average rate of fetching words from the main memory can be increased.



(a) Consecutive words in a module

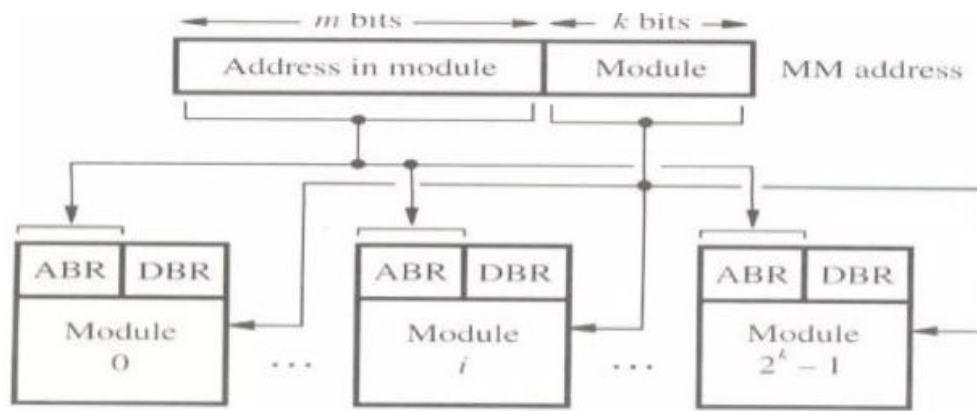
Consecutive words are placed in a module.

High order k bits of a memory Address determine the module.

Low order m bits of a memory Address determine the word within a module.

When a block of words is transferred from main memory to cache, only one module is busy at a time.





(b) Consecutive words in consecutive modules

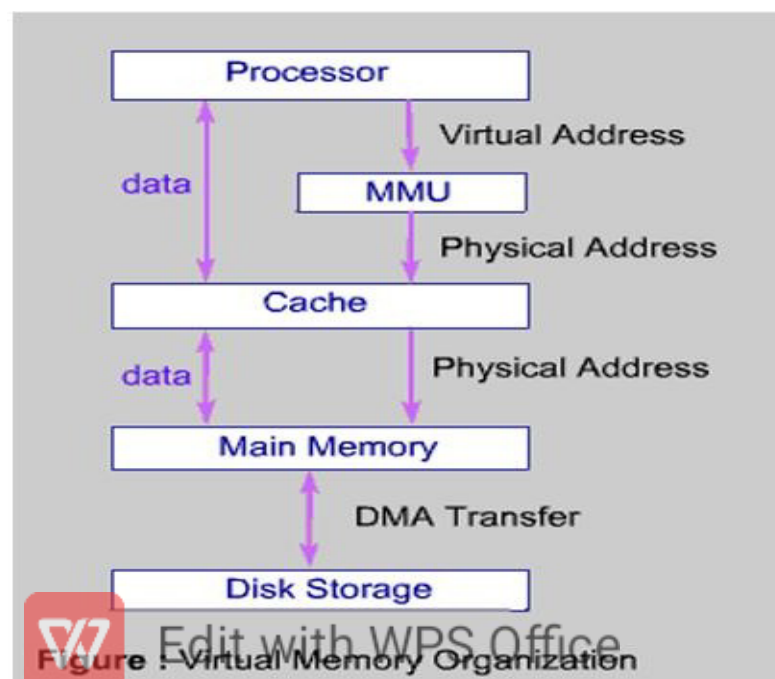
Consecutive words are located in consecutive modules.

Consecutive Addresses can be located in Consecutive modules.

While transferring a block of data, several memory modules can be kept busy at the same time.

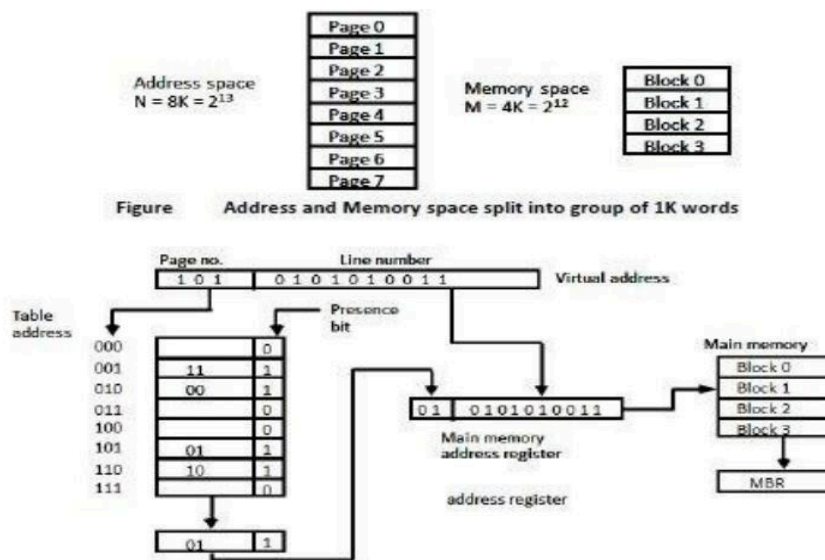
9(a) What is Virtual Memory? Discuss how address mapping using pages.

1. Virtual Memory was introduced in the system in order to increase the size the size of memory.
2. A hardware unit called Memory Management Unit (MMU) translates Virtual addresses into physical addresses.
3. If CPU wants data from main memory and it is not present in main memory then MMU causes operating system to bring the data into the Memory from disk.
4. As the disk limit is beyond the main memory address, the desired data address has to be translated from Virtual to physical address. MMU does the address translation.



Address mapping using pages:

- The table implementation of the address mapping is simplified if the information in the address space. And the memory space is each divided into groups of fixed size.
- Moreover, The physical memory is broken down into groups of equal size called blocks, which may range from 64 to 4096 words each.
- The term page refers to groups of address space of the same size.
- Also, Consider a computer with an address space of 8K and a memory space of 4K.
- If we split each into groups of 1K words we obtain eight pages and four blocks as shown in the figure.
- At any given time, up to four pages of address space may reside in main memory in any one of the four blocks.



9(b) Compare Cache and virtual memory?

S.No	Cache Memory	Virtual Memory
1	Cache Memory is the high speed of computer memory that reduces the access time of files or documents from the main memory.	Virtual Memory is a logical unit of computer memory that increases the capacity of main memory by storing or executing programs of larger size than the main memory in the computer system.
2	Cache Memory is defined as a memory unit in a computer system.	Virtual Memory is not defined as a memory unit.
3	Its size is very small as compared to Virtual Memory.	Its size is very large as compared to the Cache Memory.
4	It is a high-speed memory as compared to Virtual Memory.	It is not a high-speed memory as compared to the Cache Memory.
5	Generally, it stores frequently used data in the cache memory to reduce the access time of files.	The virtual memory keeps those data or programs that may not completely be placed in the main memory.
6	Cache Memory is controlled by the hardware of a system.	Whereas the virtual memory is control by the Operating System (OS).
7	It does not require a mapping structure to access the files in Cache Memory.	It requires a mapping structure to map the virtual address with a physical address.

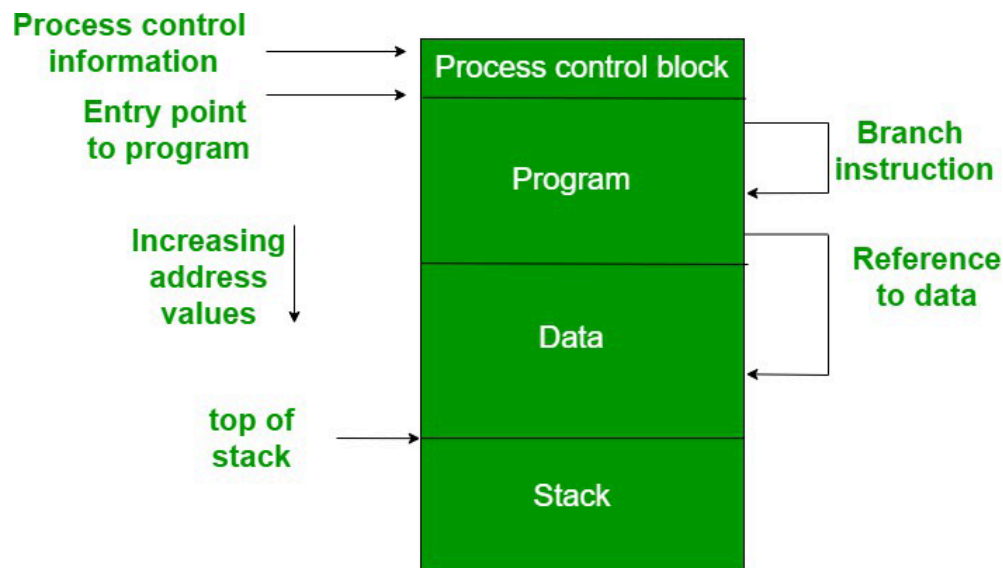
10(a) Describe in detail about the memory management requirements.

Memory Management Requirements: Memory management keeps track of the status of each memory location, whether it is allocated or free. It allocates the memory dynamically to the programs at their request and frees it for reuse when it is no longer needed. Memory management meant to satisfy some requirements that we should keep in mind.

The requirements of memory management are:

1. **Relocation:** The available memory is generally shared among a number of process in a multiprogramming system, so it is not possible to known in advance which other programs will be resident in main memory at the time of execution of this program. Swapping the active processes in and out of the main memory enables the operating system to have a larger pool of ready to execute process.





2. **Protection:** There is always a danger when we have multiple programs at the same time as one program may write to the address space of another program. So, every process must be protected against unwanted interference when other process tries to write in a process whether accidental or incidental. Between relocation and protection requirement a trade off occurs as the satisfaction of relocation memory requirement increases the difficulty of satisfying the protection management.

3. **Sharing:** A protection mechanism must have to allow several processes to access the same portion of main memory. Allowing each processes access to the same copy of the program rather than have their own separate copy has an advantage.

4. **Logical Organization:** Main memory is organized as linear or it can be a one-dimensional address space which consists of a sequence bytes or words. Most of the programs can be organized into modules, some of those are unmodifiable and some of those contain data that can be modified. To effectively deal with a user program, the operating system and computer hardware must support a basic module to provide the required protection and sharing.

5. **Physical Organization:** The structure of computer memory has two levels referred to as main memory and secondary memory. Main memory is relatively very fast and costly as compared to the secondary memory. Main memory is volatile. Thus, secondary memory is provided for storage of data on a long-term basis while the main memory holds currently used programs. The major system concern between main memory and secondary memory is the flow of information and it is impractical for programs to understand this.

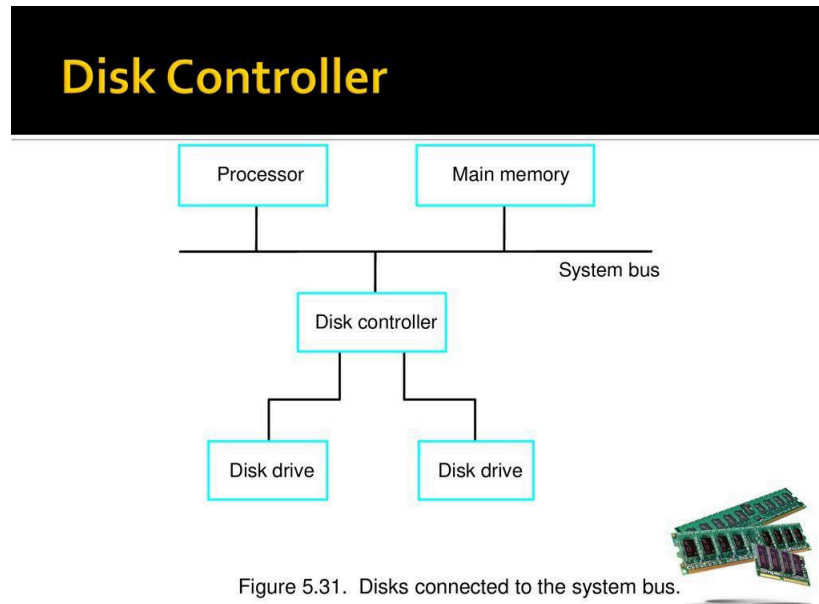
10(b) Compare various types of secondary storage systems.

Secondary storage refers to non-volatile storage devices that provide data storage for the long term, unlike primary memory (RAM) which is temporary and volatile. Secondary storage systems are essential for storing large volumes of data that are not required immediately by the processor. These systems vary in terms of speed, capacity, cost, and accessibility. Below is a comparison of various types of secondary storage systems:



1. **Magnetic hard disk:** A magnetic disk is a type of secondary memory that is a flat disc covered with a magnetic coating to hold information. It is used to store various programs and files. The polarized information in one direction is represented by 1 and vice versa. The direction is indicated by 0.

Magnetic disks are less expensive than RAM and can store large amounts of data, but the data access rate is slower than main memory because of secondary memory. Data can be modified or can be deleted easily in the magnetic disk memory. It is also a random access to data.



2. **Magnetic Tape Systems:** Magnetic tapes are suited for off time storage of large amounts of data. They are typically used for backup purposes and for archival storage. Magnetic tape recording uses the same principle as magnetic disks. The main difference is that the magnetic film is deposited on a very thin 0.5- or 0.25-inch-wide plastic tape seven or nine bits are recorded in parallel across the width of the tape, perpendicular to the direction of motion. A separate read/write head is provided for each bit position on the tape, so that all bits of a character can be read or written in parallel. One of the character bits is used as a parity bit.

Magnetic Tape Systems

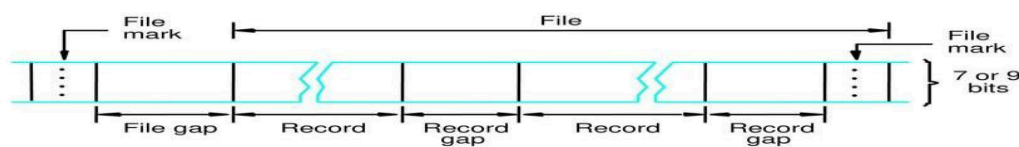


Figure 5. Organization of data on magnetic tape.

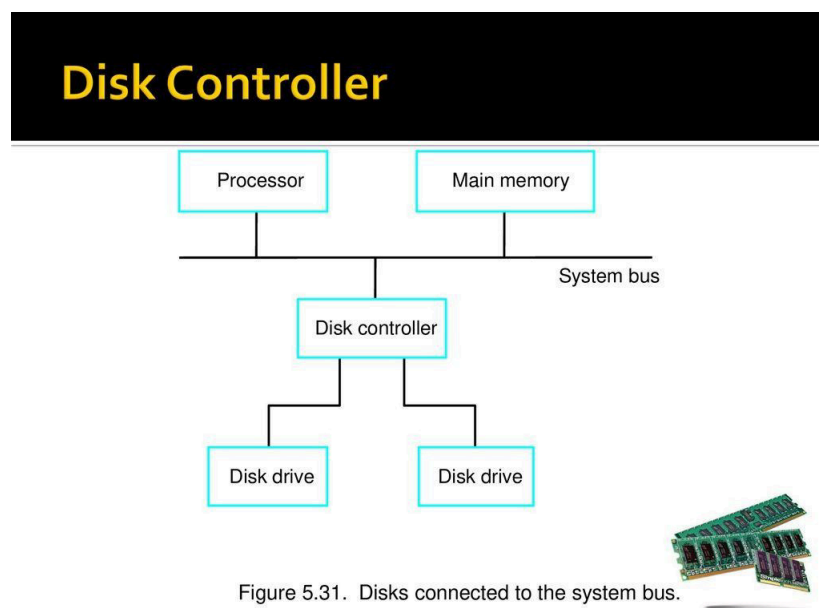


11(a) Describe the secondary storage and explain with a neat block diagram.

Secondary Storage: Secondary storage, also known as auxiliary storage or external storage, refers to non-volatile storage devices that are used to store data permanently or for extended periods of time. Unlike primary storage (RAM), which is temporary and volatile, secondary storage retains data even when the power is turned off. It is used for long-term data storage, such as the operating system, applications, files, and data that need to be accessed over time.

1. Magnetic hard disk: A magnetic disk is a type of secondary memory that is a flat disc covered with a magnetic coating to hold information. It is used to store various programs and files. The polarized information in one direction is represented by 1 and vice versa. The direction is indicated by 0.

Magnetic disks are less expensive than RAM and can store large amounts of data, but the data access rate is slower than main memory because of secondary memory. Data can be modified or can be deleted easily in the magnetic disk memory. It is also a random access to data.



2. Magnetic Tape Systems: Magnetic tapes are suited for off time storage of large amounts of data. They are typically used for backup purposes and for archival storage. Magnetic tape recording uses the same principle as magnetic disks.

The main difference is that the magnetic film is deposited on a very thin 0.5- or 0.25-inch-wide plastic tape seven or nine bits are recorded in parallel across the width of the tape, perpendicular to the direction of motion. A separate read/write head is provided for each bit position on the tape, so that all bits of a character can be read or written in parallel. One of the character bits is used as a parity bit.



Magnetic Tape Systems

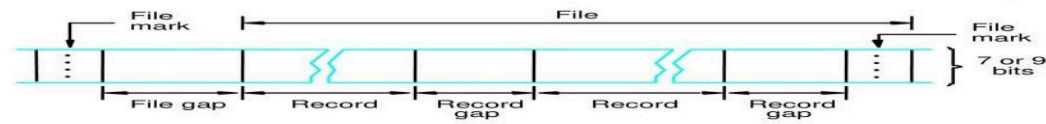


Figure 5. Organization of data on magnetic tape.

11(b) Compare Cache and Auxiliary memories?

Feature	Cache Memory	Auxiliary Memory
Purpose	Fast access to frequently used data	Permanent storage of large data
Speed	Very fast (near CPU speed)	Slow (milliseconds to seconds)
Size	Small (KB to MB)	Large (GB to TB)
Location	Close to the CPU	External or inside the system
Volatility	Volatile (loses data on power-off)	Non-volatile (data persists)
Cost	Expensive per unit	Relatively cheaper per unit
Function	Temporary storage for fast data access	Permanent storage for operating systems, apps, files
Type of Data Stored	Recently accessed data/instructions	Operating systems, applications, and user data





SIDDHARTH INSTITUTE OF ENGINEERING & TECHNOLOGY

(AUTONOMOUS)

(Approved by AICTE, New Delhi & Affiliated to JNTUA, Ananthapuramu)

(Accredited by NBA for Civil, EEE, Mech., ECE & CSE)

(Accredited by NAAC with 'A+' Grade)

Puttur -517583, Tirupati District, A.P. (India)

UNIT- V PART

-A (2 MARKS)

1(a) Define debugging?

Debugging is the process of finding and fixing errors or bugs in the source code of any software. When software does not work as expected, computer programmers study the code to determine why any errors occurred.

(b) What is interrupt and classify?

An interrupt is a signal from a device attached to a computer or from a program within the computer that requires the operating system to stop and figure out what to do next. Interrupts can be classified into hardware interrupts, triggered by external devices, and software interrupts, initiated by program instructions. Hardware interrupts include timers and I/O devices, while software interrupts include system calls and exceptions.

(c) Classify interface circuits?

An I/O interface consists of circuits which connect an, I/O device to a computer bus. As shown in Figure on one side of the interface we have the bus signals for address, data and control. On the other side we have a data path with its associated controls to transfer data between the interface and the I/O device. The interface can be classified as serial interface or parallel interface.

(d) What are the examples of processor?

The different types of processors used in computers include Central Processing Units (CPUs), Graphics Processing Units (GPUs), Multi-Core Processors, Microprocessors and Quantum Processors. Other types also include Digital Signal Processors (DSPs) and Application-Specific Integrated Circuit (ASIC).

(e) What is the need of buses and classify the bus structure?

A bus structure in a computer organization is essentially a shared communication pathway that connects multiple components of the computer system. It enables the exchange of information between parts such as the central processing unit (CPU), memory, and input/output devices.



Edit with WPS Office

PART-B (10 MARKS)

2. Explain how to access input and output devices in detail?

Input or output devices that are connected to computer are called peripheral devices. These devices are designed to read information into or out of the memory unit command from the CPU and are considered to be the part of computer system. These devices are also called peripherals. There are three types of peripherals:

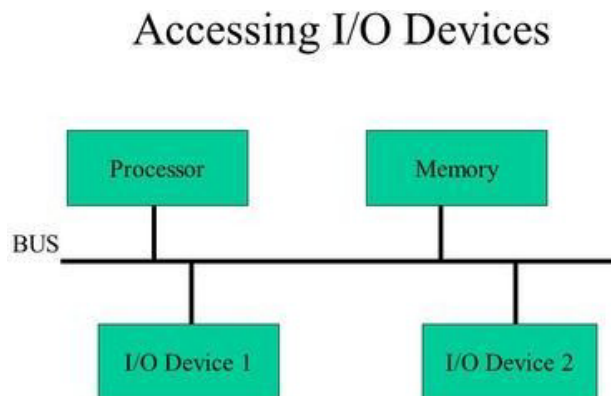
1. Input peripherals: Allows user input, from the outside world to the computer.

Example: Keyboard, Mouse etc.

2. Output peripherals: Allows information output, from the computer to the outside world. Example: Printer, Monitor etc

3. Input-Output peripherals: Allows both input (from outside world to computer) as well as, output (from computer to the outside world). Example: Touch screen etc.

Accessing I/O Devices: A simple arrangement to connect I/O devices to a computer is to use a single bus arrangement as shown below.

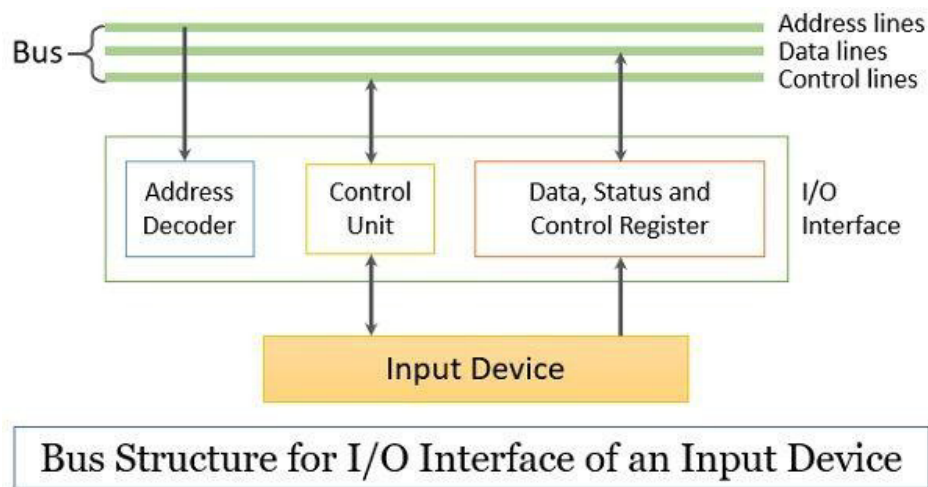


1. The bus enables all the devices connected to it exchange information.
2. Typically, the bus consists of three set of lines used to carry address, data and control signals.
3. Each I/O device is assigned a unique set of addresses.

I/O Interface for an Input device:

The address decodes the data and status registers and the circuitry required to coordinate I/O transfers constitute the interface circuit. The below figure shows the hardware required to connect on I/O device to the bus.





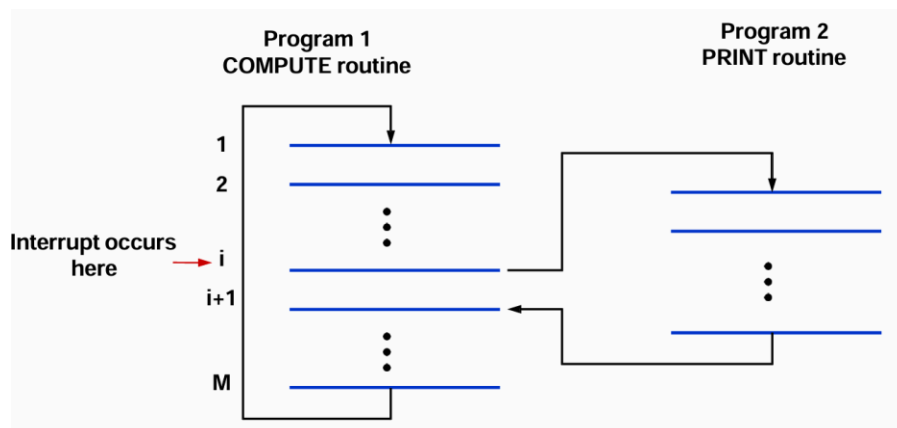
1. The address decoder enables the device to recognize its address when this address appears on the address lines.
2. The data register holds the data being transferred.
3. The status register contains information relevant to the operation of the I/O device.

3. Explain the interrupts in input/output organization?

Interrupts:

- To avoid the processor being not performing any useful computation a hardware signal called an interrupt to the processor can do it. At least one of the bus control lines called an interrupt request line is usually dedicated for this purpose.
- An integer interrupt service routine usually is needed and is executed when an interrupt request is issued.
- On the other hand, the processor must inform the device that its request has been recognized so that it may remove its interrupt-request signal. An interrupt-acknowledge signal serves this function

Example:



When building a computer based on these processors, the designer has the option of connecting I/O devices to use the special I/O address space.



Interrupt Service Routine and Subroutine:

- Treatment of an interrupt-service routine is very similar to that of a subroutine
- An important departure from the similarity should be noted
- Before executing the interrupt-service routine, any information that may be altered during the execution of that routine must be saved. This information must be restored before the interrupted program is resumed

Interrupt Latency:

- The information that needs to be saved and restored typically includes the condition code flags and the contents of any registers used by both the interrupted program and the interrupt-service routine
- Saving registers also increases the delay between the time an interrupt request is received and the start of execution of the interrupt-service routine. The delay is called interrupt latency

Identifying the Interrupting device:

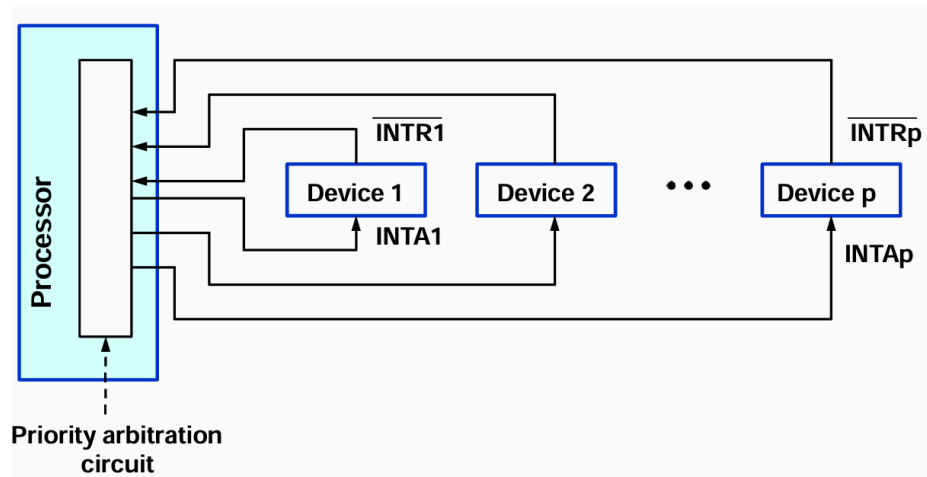
- The simplest way to identify the interrupting device is to have the interrupt-service routine poll all the I / O devices connected to the bus.
- A device requesting an interrupt may identify itself directly to the processor. Then, the processor can immediately start executing the corresponding interrupt-service routine. This is called vectored interrupts.
- An interrupt request from a high-priority device should be accepted while the processor is servicing 18 another request from a lower-priority device

Interrupt Priority:

- The processor's priority is usually encoded in a few bits of the processor status word. It can be changed by program instructions that write into the program status register (PS). These are privileged instructions, which can be executed only while the processor is running in the supervisor mode.
- The processor is in the supervisor mode only when executing operating system routines. It switches to the user mode before beginning to execute application program.

Implementation of Interrupt Priority: An example of the implementation of a multiple priority scheme





4(a) Explain the interrupt Nesting?

Interrupt nesting refers to the ability of a microprocessor or microcontroller to handle multiple interrupts in a nested or hierarchical manner. In simpler terms, it allows an interrupt that is currently being processed to be interrupted by another, higher-priority interrupt. The concept of interrupt nesting is essential in real-time systems where high-priority tasks must be handled as soon as they occur, regardless of what is currently being processed.

Key Concepts of Interrupt Nesting:

1. **Interrupt Priority:** Interrupts are typically assigned different priority levels. If a higher- priority interrupt occurs while a lower-priority interrupt is being serviced, the system can "nest" the higher-priority interrupt, suspending the lower-priority one temporarily. Once the higher-priority interrupt is handled, the lower-priority interrupt will resume from where it was interrupted.
2. **Interrupt Service Routine (ISR):** An Interrupt Service Routine (ISR) is the code that runs when an interrupt occurs. In a nested interrupt system, each ISR has its entry and exit procedures. When an interrupt occurs, the current execution state is saved (like the CPU registers), and control is transferred to the ISR. If a higher-priority interrupt occurs, the current ISR can be interrupted, and the state of the current ISR is saved. Once the higher- priority ISR completes, the processor restores the state of the previous ISR to continue processing it.
3. **Interrupt Flag and Masking:** To manage interrupt nesting, interrupt controllers or hardware mechanisms are typically used. These systems may use interrupt flags or masks to control which interrupts are allowed to pre-empt others. For example:
 - o An interrupt flag indicates if a specific interrupt is pending.
 - o Interrupt masking allows certain interrupts to be blocked temporarily to avoid unintentional interruptions.
4. **Context Saving and Restoration:** When an interrupt occurs, the CPU's current execution context (such as registers, flags, and other important values) must be

saved before servicing the interrupt. If a higher-priority interrupt occurs during the execution of a lower-priority ISR, the context of the lower-priority ISR is saved so that it can be resumed after the higher-priority interrupt is processed. This process is known as context saving and restoration.

5. **Stack Management:** In nested interrupts, the stack is heavily used to save and restore the execution context. Each time an interrupt occurs, the processor pushes the current context (e.g., registers, program counter) onto the stack. When the interrupt is finished, the context is popped from the stack to resume the previous state.

4(b) Explain about interrupt service routine (ISR)?

Interrupt service routine (ISR) is actually a call-back function (program) in the case of software or device driver (I/O device) in the case of hardware. When an interrupt is acknowledged by the processor, the routine or program which is running currently gets paused or interrupted, and the ISR program gets executed. An interrupt is a signal to the processor emitted by hardware or software indicating an event that needs immediate attention. Whenever an interrupt occurs, the controller completes the execution of the current instruction and starts the execution of an Interrupt Service Routine (ISR) or Interrupt Handler.

- An interrupt service routine (ISR) is a software routine that hardware invokes in response to an interrupt.
- ISRs examine an interrupt and determine how to handle it.
- ISRs handle the interrupt and then return a logical interrupt value.
- Its central purpose is to process the interrupt and then return control to the main program.
- An ISR must perform very fast to avoid slowing down the operation of the device and the operation of all lower priority ISRs.
- As in procedures, the last instruction in an ISR should be met.

A small program or a routine that when executed, services the corresponding interrupting source is called an ISR.

1. Saving the processor context

Because the ISR and main program use the same processor registers, it is the responsibility of the ISR to save the processor's registers before beginning any processing of the interrupt. The processor context consists of the instruction pointer, registers, and any flags. Some processors perform this step automatically.

2. Acknowledging the interrupt



The ISR must clear the existing interrupt, which is done either in the peripheral that generated the interrupt, in the interrupt controller, or both.

3. Restoring the processor context

After interrupt processing, in order to resume the main program, the values that were saved prior to the ISR execution must be restored. Some processors perform this step automatically.

5 Give detailed notes on DMA transfers with neat sketch.

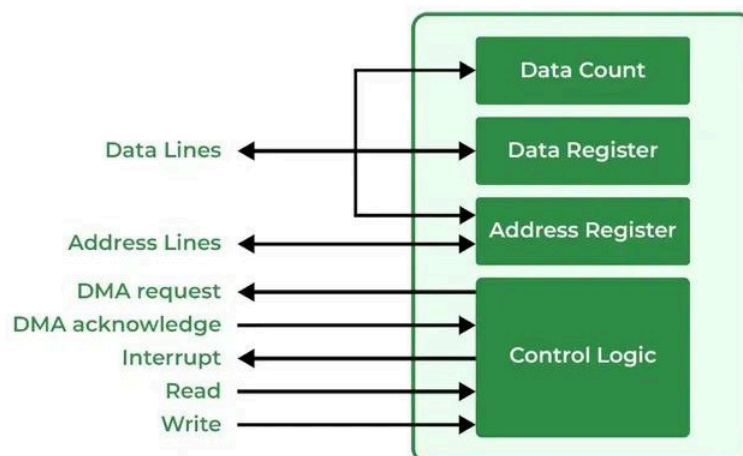
In modern computer systems, transferring data between input/output devices and memory can be a slow process if the CPU is required to manage every step. To address this, a Direct Memory Access (DMA) Controller is utilized. A Direct Memory Access (DMA) Controller solves this by allowing I/O devices to transfer data directly to memory, reducing CPU involvement. This increases system efficiency and speeds up data transfers, freeing the CPU to focus on other tasks. DMA controller needs the same old circuits of an interface to communicate with the CPU and Input/Output devices.

DMA Controller: Direct Memory Access (DMA) uses hardware for accessing the memory, that hardware is called a DMA Controller. It has the work of transferring the data between Input Output

devices and main memory with very less interaction with the processor. The direct Memory Access Controller is a control unit, which has the work of transferring data.

DMA Controller in Computer Architecture:

DMA Controller is a type of control unit that works as an interface for the data bus and the I/O Devices. As mentioned, DMA Controller has the work of transferring the data without the intervention of the processors, processors can control the data transfer. DMA Controller also contains an address unit, which generates the address and selects an I/O device for the transfer of data. Here we are showing the block diagram of the DMA Controller.



There are four popular types of DMA.

- Single-Ended DMA
- Dual-Ended DMA
- Arbitrated-Ended DMA
- Interleaved DMA

Single-Ended DMA: Single-Ended DMA Controllers operate by reading and writing from a single memory address. They are the simplest DMA.

Dual-Ended DMA: Dual-Ended DMA controllers can read and write from two memory addresses. Dual-ended DMA is more advanced than single-ended DMA.

Arbitrated-Ended DMA: Arbitrated-Ended DMA works by reading and writing to several memory addresses. It is more advanced than Dual-Ended DMA.



Interleaved DMA: Interleaved DMA are those DMA that read from one memory address and write from another memory address.

Working of DMA Controller

The DMA controller registers have three registers as follows.

- **Address register** – It contains the address to specify the desired location in memory.
- **Word count register** – It contains the number of words to be transferred.
- **Control register** – It specifies the transfer mode.

6(a) Distinguish between Centralized arbitration and Distributed arbitration?

S.No	CENTRALIZED ARBITRATION	DISTRIBUTED ARBITRATION
1	All the component modules are deployed at same place.	Whereas in distributed they are deployed at multiple places.
2	Highly vulnerable and hence not robust against DDoS attacks.	Less vulnerable and hence not robust against DDoS attacks.
3	No cooperation and communication framework required.	Cooperation among various modules and proper communication framework required.
4	Lesser resources are available for fighting against the attacks.	More resources are available for fighting against the attacks.
5	Mostly deployed at victim site	Deployed at victim-core, throughout the internet and victim-core

6(b) What is the need of BUS and explain different types in detail?

A bus in computing is a system of pathways used to transmit data between components, such as the CPU, memory, and input/output devices. It is a crucial part of a computer's architecture because it enables the transfer of data and instructions between different hardware parts. Without a bus, these components would not be able to communicate efficiently, and the computer system would not function as intended.

Need for a Bus

1. **Data Communication:** A bus facilitates the transfer of data between the CPU, memory, and peripherals. It allows different parts of the computer to exchange information without needing separate connections for each device.



2. **Cost and Space Efficiency:** Using a bus to connect components reduces the complexity, cost, and physical space requirements of the system. If each component needed individual connections, it would require significantly more wiring and circuitry.
3. **Parallel Processing:** A bus allows for parallel communication, which means that multiple devices can exchange data simultaneously, leading to better system performance.
4. **Simplification of the Design:** Instead of having multiple direct connections between all components, a bus architecture simplifies the design by providing a central communication pathway that can be shared by many components.

Different types of Bus:

A bus system can be classified into different types based on its role and usage in a computer system. The main types are:

1. **Data Bus:** The data bus carries the actual data being transferred between the CPU, memory, and other peripherals. The width of the data bus (in bits) determines how much data can be transmitted at once. For example, a 32-bit data bus can transfer 32 bits of data in a single operation. It is usually bidirectional, meaning it can carry data in both directions (from CPU to memory or I/O devices, and vice versa).
2. **Address Bus:** The address bus is used to carry the address of the memory location or I/O port that the CPU wants to read from or write to. It is typically unidirectional (from the CPU to memory or I/O devices) because the CPU specifies the address, and no address is returned by the memory or devices. The width of the address bus determines how much memory or how many I/O devices can be addressed. For instance, a 32-bit address bus can address 2^{32} (4 GB) of memory.
3. **Control Bus:** The control bus carries control signals that direct the operations of the CPU, memory, and other devices. These signals can include read/write signals, clock signals, interrupt requests, etc. Unidirectional or Bidirectional: Some control lines may be unidirectional, while others may be bidirectional. Control bus signals ensure that components work in sync. For example, when the CPU needs to read data from memory, the control bus sends a read signal to memory.

7. Types of interfacing circuits and explain

it? Interface Circuits:

- An I/O interface consists of the circuitry required to connect an I/O device to a computer bus.
- On one side of the interface, we have the bus signals for address, data and control.
- On the other side, we have a data path with its associated controls to transfer

data between the interface and the I/O device.

- This side is called a port, and it can be classified as either a parallel port or a serial port

Parallel Port: A parallel port transfers data in the form of a number of bits, typically 8 or 16 simultaneously to or from the device. In case of a parallel port the connection between the device and the computer uses a multipin connector and a cable with as many wires. The circuits of either end are relatively simple. This arrangement is suitable for devices that are physically close to the computer.

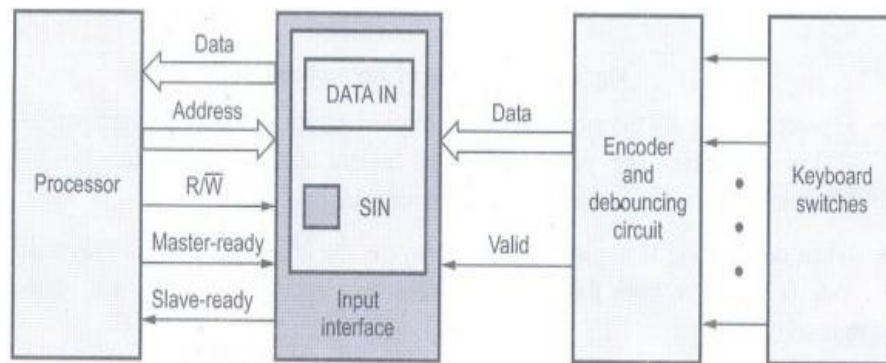
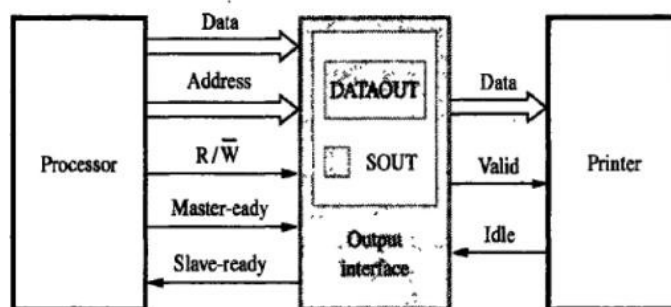


Fig. 8.9.1 Hardware components for connecting a keyboard to a processor

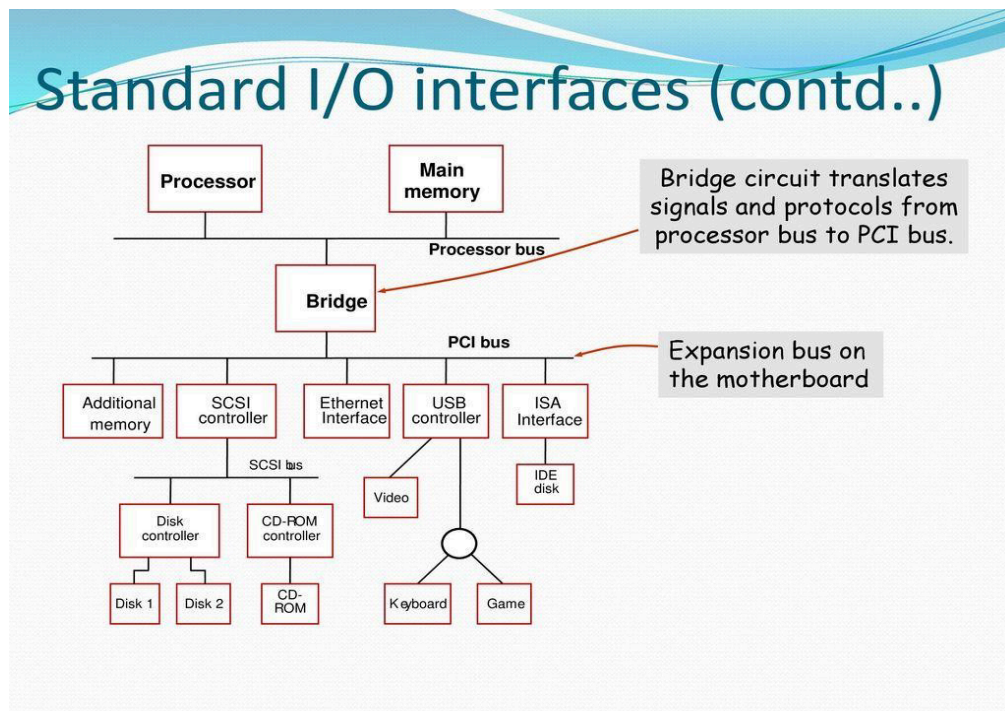
Serial Port: A Serial port transmits and receives data one bit at a time. Communication with bus is the same for both formats. The serial format is much more convenient and cost effective where longer cables are needed. Because serial interfaces play a vital role in connecting I/O devices, several widely used standards have been developed.



8. Explain the standard I/O devices?

It is observed that I/O devices fitted with an interface circuit suitable for one computer may not be usable with other computers. A different interface may have to be designed for every combination of I/O device and computer, resulting in many different interfaces. The most practical solution is to develop standard interfaces. A typical computer, for example includes a printed circuit board called the mother board which consists of processor chip, the main memory and some I/O interfaces. In this section we

present three widely used bus standards which are used in a typical computer system as shown below



1. Peripheral component Interconnect (PCI) Bus: The PCI was developed as a low-cost bus that is truly processor independent. Devices connected to the PCI bus appear as the processor as if they were connected directly to the processor bus. An important feature of PCI bus is a plug and play capability for connecting I/O devices.

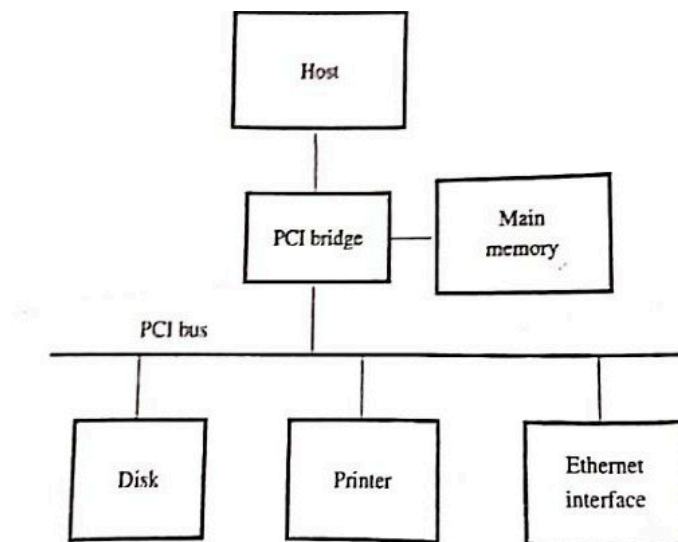
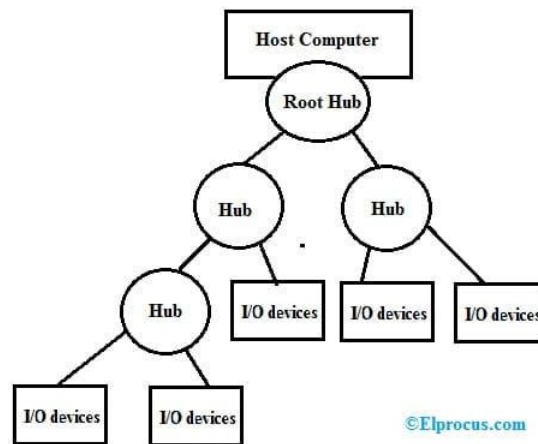


Figure 4.39 Use of a PCI bus in a computer system.

2. SCSI Bus: SCSI Bus stands for Small Computer System Interface. It was defined by American national standards Institute (ANSI). A SCSI bus may have eight data lines, in which case it is called a narrow bus and transfers data one byte at a time. Alternatively, a whole SCSI bus has 16 data lines and transfers data 16 bit at a time. The bus may use single ended transmission, where each signal uses a one wire with a common ground return for all signals.

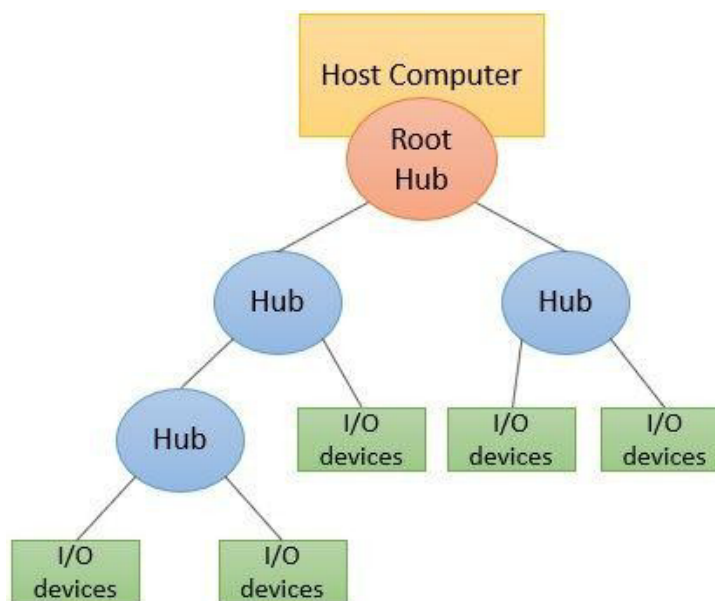


3. Universal Serial Bus (USB): A modern computer system is likely to involve a wide variety of devices such as keyboards, microphones, cameras, speakers etc. A key requirement in such an environment is the availability of a simple, low-cost mechanism to connect these devices to the computer and an important recent development in this regard is the introduction of USB. The USB supports two speeds of operation called low speed and full speed.



9. Draw the USB architecture and explain it

USB Architecture: To accommodate a large number of devices that can be added or removed at any time, the USB has the tree structure shown below.



Universal Serial Bus Tree Structure

- Each node of the tree has a device called a hub, which acts as an intermediate transfer point between the host computer and the I/O devices.
- At the root of the tree, a root hub connects the entire tree to the host computer.

- The leaves of the tree are the I/O devices: a mouse, a keyboard, a printer, an Internet connection, a camera, or a speaker.
- If I/O devices are allowed to send messages at any time, two messages may reach the hub at the same time and interfere with each other.
- For this reason, the USB operates strictly on the basis of polling.
- A device may send a message only in response to a poll message from the host processor.
- Each device on the USB, whether it is a hub or an I/O device, is assigned a 7-bit address.
- The root hub of the USB, which is attached to the processor, appears as a single device.
- The host software communicates with individual devices by sending information to the root hub, which it forwards to the appropriate device in the USB tree.
- When a device is first connected to a hub, or when it is powered on, it has the address 0. Periodically, the host polls each hub to collect status information and learn about new devices that may have been added or disconnected.
- When the host is informed that a new device has been connected, it reads the information in a special memory in the device's USB interface to learn about the device's capabilities.
- It then assigns the device a unique USB address and writes that address in one of the device's interface registers.
- It is this initial connection procedure that gives the USB its plug-and-play capability.

10(a) Draw the PCI bus architecture and explain its operation?

The **PCI (Peripheral Component Interconnect) bus** is a system that allows peripheral devices (such as network cards, graphics cards, and storage controllers) to communicate with the central processing unit (CPU) and memory. Below is an explanation of the **PCI bus architecture** and its **operation**

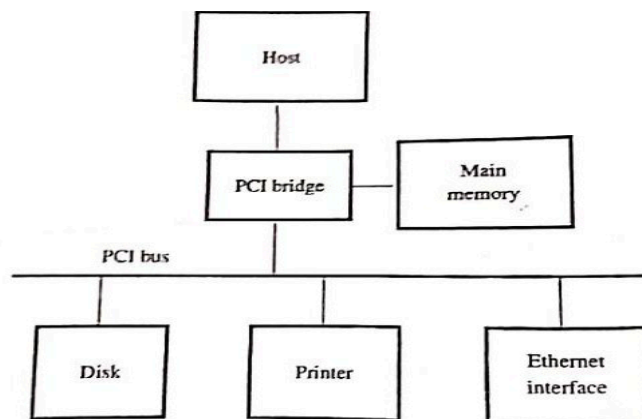


Figure 4.39 Use of a PCI bus in a computer system.

Operation of PCI Bus

The operation of the PCI bus involves several stages, from initialization to data transfer. Here's a step-by-step overview of the process:

1. Bus Initialization:

- When the system is powered on, the PCI controller initializes the bus by performing a configuration cycle.
- The PCI devices are enumerated, meaning each device is assigned a unique **bus number**, **device number**, and **function number** for addressing.

2. Data Transfer Process:



- o **Bus Mastering:** PCI supports a mechanism known as **bus mastering**, where devices (not just the CPU) can take control of the bus to transfer data to and from memory.
 - o **Read/Write Cycle:** During a read or write cycle, the bus master (device or CPU) asserts the appropriate control signals to begin the transfer.
 - **Read Cycle:** The bus master sends an address and signals a read operation. The memory or device responds by placing the data on the bus.
 - **Write Cycle:** The bus master sends an address and signals a write operation, then places the data onto the bus to be written to the specified memory location or device.
3. **Arbitration:**
- o Since multiple devices can potentially want to use the bus at the same time, the PCI bus uses an **arbitration process** to determine which device gets control of the bus.
 - o The **Arbitration Logic** handles this, ensuring that only one device can use the bus at any given time, preventing conflicts.
4. **Data Transfer:**
- o Once the bus is granted to a device, the data transfer begins. The device places the data on the **data lines**, and the other components (CPU or memory) accept or send data accordingly.
 - o The **control lines** handle synchronization and the type of transfer (read or write).
5. **Interrupt Handling:**
- o PCI devices can generate interrupts to signal the CPU that they need attention. This allows peripherals to notify the CPU about events that require processing (e.g., completion of a data transfer).
 - o Interrupts are handled by a **central interrupt controller**.

10(b) Explain about SCSI BUS in detail?

It is a standard bus defined by the American National Standards Institute (ANSI). A controller connected to a SCSI bus is an initiator or a target. The processor sends a command to the SCSI controller, which causes the following sequence of events to take place:

- o The SCSI controller contends for control of the bus (initiator).
- o When the initiator wins the arbitration process, it selects the target controller and hands over control of the bus to it.
- o The target starts an output operation. The initiator sends a command specifying the required read operation.
- o The target sends a message to the initiator indicating that it will temporarily suspend the connection between them. Then it releases the bus.
- o The acronym SCSI stands for Small Computer System Interface. It refers to a standard bus defined by the American National Standards Institute (ANSI) under the designation X3.131 .
- o In the original specifications of the standard, devices such as disks are connected to a computer via a 50-wire cable, which can be up to 25 meters in length and can transfer data at rates up to 5 megabytes/s.
- o The SCSI bus standard has undergone many revisions, and its data transfer capability has increased very rapidly, almost doubling every two years. SCSI-2 and SCSI-3 have been defined, and each has several options.
- o A SCSI bus may have eight data lines, in which case it is called a narrow bus and transfers data one byte at a time.
- o Alternatively, a wide SCSI bus has 16 data lines and transfers data 16 bits at a time.

- o There are also several options for the electrical signaling scheme used.
- o Devices connected to the SCSI bus are not part of the address space of the processor in the same way as devices connected to the processor bus.
- o The SCSI bus is connected to the processor bus through a SCSI controller.
- o This controller uses DMA to transfer data packets from the main memory to the device, or vice versa.

11(a) Compare data, address and control buses?

Data Bus	Address Bus	Control Bus
Transfers actual data	Transfers memory addresses	Carries control signals (read, write, etc.)
Bidirectional	Unidirectional	Unidirectional
Determines the amount of data transferred at a time	Determines the memory addressable space	Varies, depends on number of control signals
Carries data to/from CPU, memory, I/O	Specifies the memory or I/O address	Signals control actions for data transfers

11(b) Explain about input and output interface

circuits? Interface Circuits:

- An I/O interface consists of the circuitry required to connect on I/O device to a computer bus.
- On one side of the interface, we have the bus signals for address, data and control.
- On the other side, we have a data path with its associated controls to transfer data between the interface and the I/O device.
- This side is called a port, and it can be classified as other a parallel port or a serial sort

Parallel Port: A parallel port transfers data in the form of a number of bits, typically 8 or 16 simultaneously to or from the device. In case of a parallel port the connection between the device and the computer uses a multipin connector and a cable with as many wires. The circuits of either end as relatively simple. This arrangement is suitable for devices that are physically close to the computer.



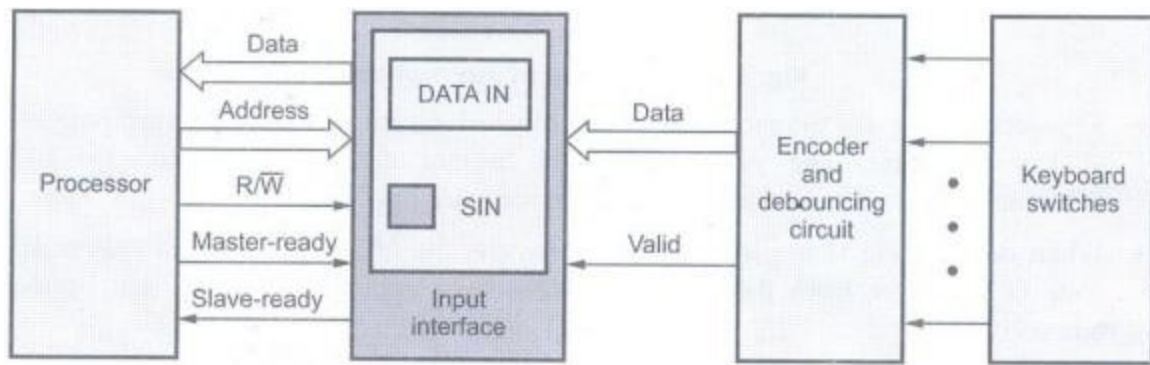


Fig. 8.9.1 Hardware components for connecting a keyboard to a processor

Serial Port: A Serial port transmits and receives data one bit at a time. Communication with bus is the same for both formats. The serial format is much more convenient and cost effective where longer cables are needed. Because serial interfaces play a vital role in connecting I/O devices, several widely used standards have been developed.

